

# 면접 파이프라인 감리 보고서

제출물 분석 · 실시간 면접 · 근거 리포트 전 구간 코드 레벨 점검

## 감리 방식

기획 문서가 아니라 **실제 소스 코드와 실행 결과**를 기준으로 점검했습니다. 모든 수치·가중치·상태 전이·테이블 컬럼은 이 저장소의 `backend/src` , `apps/` 에서 직접 읽어 확인한 값입니다. 문서와 코드가 다를 때는 코드를 사실로 봤습니다.

기업의 평가 기준으로 질문하고, 지원자의 실제 산출물과 실제 답변에서 채용 판단 근거를 만드는 AI 구조화 면접 SaaS. **AI는 합격·불합격을 결정하지 않으며, 모든 평가 항목은 최종 답변 턴으로 역추적됩니다.** 이 감리는 그 불변식이 코드에서 실제로 지켜지는지를 확인합니다.

|         |        |         |         |
|---------|--------|---------|---------|
| 면접 전 분석 | 실시간 면접 | 후처리·리포트 | 영상 분석   |
| 동작      | 동작     | 부분 구현   | 미구현(의도) |

## 기준 커밋

7c998be

main · 워킹 트리 클린

## 감리 일자

2026-08-16

코드 레벨 전수 점검

## 백엔드 테스트

193 passed

1 skipped · 7.13s

## 발견 사항

9건

차단 1 · 운영 전 필수 4 · 개선 4

## 00 목차

1~2장은 결론과 이번 정리 작업 내역, 3~4장은 구조와 데이터 모델, 5~10장은 파이프라인 실동작, 11~14장은 판정입니다.

### PART 1 · 결론과 작업 내역

|    |                            |    |
|----|----------------------------|----|
| 01 | 감리 요약                      | 03 |
|    | 무엇을 봤고, 무엇이 맞고, 무엇이 비어 있는가 |    |
| 02 | 0~5단계 정리 작업 내역             | 04 |
|    | 커밋 5개 · 무엇을 왜 지웠는지의 근거     |    |

### PART 2 · 구조와 데이터 모델

|    |                               |    |
|----|-------------------------------|----|
| 03 | 시스템 토폴로지                      | 05 |
|    | 2개 SPA · 4개 레인 · 워커 큐 4개      |    |
| 04 | ERD — 전체 50개 테이블              | 06 |
|    | 레인별 ERD 5장 + 테이블별 역할 표        |    |
| 05 | ID 계보                         | 12 |
|    | 기준 버전 ID가 초대→전략→세션→리포트까지 동일한지 |    |

### PART 3 · 파이프라인 실동작

|    |                                        |    |
|----|----------------------------------------|----|
| 06 | 시퀀스 1 — 면접 전 제출물 분석                    | 13 |
|    | 업로드 → 소유권 판정 → 검증 타킷 생성                |    |
| 07 | 하이브리드 검색(RAG)                          | 15 |
|    | 벡터 0.55 / 어휘 0.30 / 심볼 0.80 / 소유권 0.15 |    |
| 08 | 시퀀스 2 — 실시간 면접 1턴                      | 16 |
|    | 음성·영상 이중 스트림 · 꼬리질문 분기 · 정책 검사         |    |
| 09 | 세션 상태 기계와 복구                           | 18 |
|    | 8개 상태 · DynamoDB 핫뷰 3단 폴백              |    |
| 10 | 시퀀스 3 — 면접 후 워커 체인                     | 20 |
|    | interview.completed → 미디어 → 리포트        |    |

### PART 4 · 쟁점과 판정

|    |                              |    |
|----|------------------------------|----|
| 11 | 영상은 어떻게 들어가는가                | 24 |
|    | 영상 "분석"은 없다 — 무결성과 근거 앵커     |    |
| 12 | 질문·꼬리질문·답변의 저장 구조            | 25 |
|    | 3개 테이블이 나눠 갖는 책임             |    |
| 13 | 발견 사항 9건과 우선순위               | 26 |
|    | 재현 조건 · 영향 · 권고 조치 · F-09 차단 |    |
| 14 | 감리 판정                        | 30 |

#### 이 문서를 읽는 방법

모든 다이어그램은 인라인 SVG이므로 인터넷 연결 없이, 인쇄해도 그대로 나옵니다. 다이어그램 안의 표기 규칙은 다음과 같습니다.

- PK 붉은 글씨 · FK 파란 글씨 · ▶ **보호 대상 텍스트**(지원자 원문·답변 원문, 로그 금지)
- ⚠ 표시는 **감리에서 발견한 미완성 지점**입니다. 13장의 발견 사항 번호와 연결됩니다.
- 실선 화살표는 코드에 실제로 존재하는 호출·외래 키, 점선은 JSON 컬럼을 통한 느슨한 참조입니다.

# 01 감리 요약

한 문장 결론: **면접 전 분석과 실시간 면접은 설계대로 동작하고, 면접 후 리포트 구간은 배선만 되어 있고 내용이 비어 있다.**

## 1.1 무엇을 점검했는가

기획서·설계서를 근거로 삼지 않고, 다음을 직접 읽어 확인했습니다.

| 점검 대상          | 확인 방법                                                                         | 결과                                                                           |
|----------------|-------------------------------------------------------------------------------|------------------------------------------------------------------------------|
| 50개 테이블의 컬럼 정의 | 4개 레인 + shared의 <code>repositories/postgres.py</code> 전수 열람                   | ERD로 재구성 완료(4장). 모든 테이블이 <code>company_id</code> 를 보유                        |
| 꼬리질문 결정 규칙     | <code>interview_plan.py:64</code> 함수 본문                                       | <code>follow_up_count &lt; max_follow_ups</code> 단일 조건. AI 판단 아님             |
| 하이브리드 검색 가중치   | <code>postgres_hybrid.py:149</code> , <code>retrieval.py:12</code>            | SQL 1차 정렬과 애플리케이션 재정렬의 가중치가 다름 — 의도된 2단 구조                                   |
| DynamoDB 핫뷰 폴백 | <code>context_reconciliation.py:45</code> 분기 3개                               | 3단 폴백 모두 구현. 실패 시 <code>degraded_modes</code> 로 노출                           |
| 영상 분석 존재 여부    | <code>rekognition.face.emotion.gaze.posture.eye_contact.facial</code> 전수 grep | <b>0건.</b> 영상 분석 기능은 존재하지 않음(11장)                                            |
| 세션 상태 전이 8개    | <code>domain/session.py:68-91</code> 전이표 + <code>transition()</code> 호출 지점 전수 | 6개는 실행됨. <code>REPORT_GENERATING</code> / <code>REVIEWABLE</code> 은 호출 지점 0건 |
| 이벤트 큐 라우팅      | <code>runtime/worker.py:51-67</code>                                          | 16개 이벤트 라우팅. <code>interview.checkpoint_changed</code> 는 발행되지만 미등록           |
| 테스트 실패         | <code>uv run --no-sync</code><br><code>pytest backend/tests</code>            | 193 passed · 1 skipped · 7.13s                                               |

## 1.2 잘 되어 있는 것 (그대로 유지 권고)

### 근거 역추적이 구조로 강제될

질문 하나마다 `question_rationales` (왜 물었는가)와 `question_source_references` (무엇을 근거로 물었는가)가 별도 행으로 남습니다. 리포트의 `evidence` 는 **지원자 최종 턴 ID와 자막 세그먼트, 영상 구간**을 모두 가리켜야만 `CONFIRMED` 가 됩니다. 검증 실패 시 조용히 통과하지 않고 `INSUFFICIENT_EVIDENCE` 로 떨어집니다.

### 환각 출처를 코드가 잘라냄

질문 생성 모델이 반환한 출처 ID 중 **그 턴에서 실제로 검색된 ID 집합에 없는 것은 버립니다** ( `interview_service.py` 의 `retrieved_by_id` 필터). 모델이 그럴듯한 출처를 만들어 내도 저장되지 않습니다. 감리에서 가장 높게 평가하는 지점입니다.

#### 역등성이 데이터베이스에 있음

웹소켓 명령은 `interview_command_results` 에 (`company_id`, `session_id`, `operation`, `idempotency_key`) 복합 PK로 결과를 저장합니다. 네트워크가 끊겨 같은 `answer.complete` 가 두 번 와도 턴은 하나만 생깁니다. 메모리 캐시가 아니라 durable 저장이라 프로세스 재시작에도 유지됩니다.

#### 테넌시가 SQL 레이어에 있음

테넌트 분리를 API 검증에만 의존하지 않고 리포지토리 쿼리의 `WHERE company_id` 로 강제합니다. 다른 회사 객체를 조회하면 403이 아니라 **404**가 나오므로 존재 여부조차 노출되지 않습니다.

### 1.3 비어 있는 것 (발견 사항 요약)

9건 중 8건은 개발 단계에서 의도적으로 미룬 것으로 보이는 미완성이지만, 1건(F-09)은 저장된 값이 틀리는 결함이며 검증을 통과해 맞는 것처럼 보입니다. 상세는 13장입니다.

| ID   | 등급      | 내용                                                                | 영향                                                       |
|------|---------|-------------------------------------------------------------------|----------------------------------------------------------|
| F-09 | 차단      | 답변 턴의 영상 구간을 턴 인덱스 = 청크 인덱스로 정함                                   | 두 번째 턴부터 시각이 틀림. 그런데 검증을 통과해 <code>CONFIRMED</code> 가 붙음 |
| F-01 | 운영 전 필수 | 리포트 생성에 모델 호출이 없음 — <code>observation</code> 이 고정 문자열             | 리포트가 실제 평가 내용을 담지 못함                                     |
| F-02 | 운영 전 필수 | <code>create_hls_job</code> 이 어디서도 호출되지 않음                        | 재생용 HLS가 만들어지지 않음. 매니페스트 키만 기록                           |
| F-03 | 운영 전 필수 | <code>REPORT_GENERATING</code> · <code>REVIEWABLE</code> 전이 호출 0건 | 세션이 <code>COMPLETED</code> 에서 멈춤. 진행 상황 표시 불가            |
| F-04 | 운영 전 필수 | <code>hot_view_sync_status</code> 가 항상 <code>PENDING</code>       | 핫뷰 동기 상태를 신뢰할 수 없음. 운영 지표로 못 씀                           |
| F-05 | 개선      | <code>interview.checkpoint_changed</code> 가 라우팅 표에 없음             | 소비 로직( <code>reconcile_event</code> )은 있는데 배선만 없음        |
| F-06 | 개선      | <code>query_vector=None</code> 으로 매 턴 임베딩 재계산                     | 턴마다 불필요한 임베딩 1회. 실시간 지연 요인                               |
| F-07 | 개선      | <code>FinalTurnRange.confidence=0.9</code> 고정값                    | 자막 신뢰도가 근거 검증에 반영되지 않음                                   |
| F-08 | 개선      | 기준×답변 턴을 <code>zip(strict=False)</code> 로 짝지움                     | 수가 다르면 뒤쪽이 조용히 누락. 기준 미평가 은폐                             |

#### 개발 단계 전제에 대한 감리 의견

F-01~F-07은 "버그"가 아니라 **완성 순서의 결과**로 보입니다. 실제로 F-01·F-02·F-03은 모두 "면접이 끝난 다음" 구간에 몰려 있고, 그 앞 구간(분석·면접)은 세부까지 구현돼 있습니다. 로컬 런타임이 `LocalDeterministicModel` · `StaticTextEmbedder` · `DeterministicSpeechToText` 로 완전 결정적으로 동작하도록 만들어져 있다는 점이 이를 뒷받침합니다 — **테스트를 먼저 안정화하고 모델 연동을 나중에 붙이는 순서**입니다.

다만 F-09와 F-08은 성격이 다릅니다. 이 둘은 미완성이 아니라 **구현된 로직이 틀린** 것이고, 둘 다 **예외도 로그도 남기지 않아** 시간이 지나면 원인을 찾기 어렵습니다. 특히 F-09는 `CONFIRMED` 판정까지 정상적으로 받으므로, 지금 닫는 것을 권고합니다.

## 02 0~5단계 정리 작업 내역

커밋 5개. 되돌림 지점은 `abb95cf` 이며 그 위 4개가 이번 정리입니다. 워킹 트리는 클린 상태입니다.

|                                     |                                   |                                    |                                         |
|-------------------------------------|-----------------------------------|------------------------------------|-----------------------------------------|
| 커밋<br><b>5</b><br>abb95cf 되돌림 지점 포함 | MAKE 타겟<br><b>40→17</b><br>23개 제거 | CI 잡<br><b>1</b><br>workspace 단일 잡 | 테스트<br><b>193</b><br>passed · 1 skipped |
|-------------------------------------|-----------------------------------|------------------------------------|-----------------------------------------|

### 2.1 커밋별 내역과 결정 근거

| 커밋      | 무엇을                                   | 규모                            | 왜 — 판단 근거                                                                                                                                                                                                     |
|---------|---------------------------------------|-------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| abb95cf | 기업 콘솔·지원자 셀 재구성 <b>기준</b>             | 233 files<br>+28,719 / -2,593 | 이번 정리의 <b>되돌림 지점</b> . 문제가 생기면 <code>git revert</code> 대상은 이 위의 4개뿐입니다.                                                                                                                                       |
| ff544b1 | CI가 전체 테스트를 돌리게 하고, 안 쓰는 의존성 제거       | 7 files<br>+57 / -103         | 기존 CI는 여러 잡으로 쪼개져 있으면서도 <b>실제로는 일부 테스트만</b> 돌렸습니다. <code>npm run test:workspace</code> 하나가 포맷·린트·타입체크·전 테스트를 포함하므로, 잡을 나누는 것은 병렬 이득 없이 캐시만 중복됐습니다. 단일 <code>workspace</code> 잡으로 합쳤습니다.                       |
| 2c386bb | 리포지토리 팬아웃 축소, 낙관적 동시성 충돌 타이핑          | 9 files<br>+300 / -138        | 목록 화면 한 번에 리포지토리 호출이 N+1로 퍼지던 지점을 한 번의 조회로 모았습니다. 포지션 버전 충돌은 <code>str</code> 비교가 아니라 타입이 있는 결과로 바꿔 <code>If-Match-Version</code> 불일치가 조용히 성공하지 않게 했습니다.                                                      |
| 70a8549 | 프론트엔드 API 응답 타입을 생성 계약에서 가져오게 변경      | 7 files<br>+83 / -229         | 두 SPA의 <code>routeAdapters.tsx</code> 가 API 응답 모양을 <b>손으로 다시 선언</b> 하고 있었습니다. 계약이 바뀌어도 프론트는 컴파일 성공하는 구조 — 가장 위험한 종류의 중복입니다. <code>components["schemas"] [...]</code> 로 교체해 계약이 바뀌면 타입체크가 깨지게 만들었습니다. 순감 146줄. |
| 7c998be | npm 스크립트나 pytest로 표현 못 하는 Make 타겟만 남김 | 5 files<br>+33 / -108         | Make 타겟 40개 중 23개가 <code>npm run ...</code> 또는 <code>pytest ...</code> 한 줄의 별칭이었습니다. 별칭은 "어느 쪽이 진짜인가"를 흐리고 CI와 로컬이 다른 경로를 타게 만듭니다. 17개만 남겼습니다.                                                                |

### 2.2 남긴 Make 타겟 17개 — 남긴 이유

기준은 하나입니다. `npm 스크립트나 pytest 한 줄로 똑같이 표현할 수 없는 것만` 남겼습니다. Makefile 첫 줄에 그 기준을 주석으로 적어 뒀습니다.

| 분류        | 타겟                                                                                                                                        | 대체 불가능한 이유                                                                                                                  |
|-----------|-------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------|
| 환경 구성     | <code>bootstrap</code> · <code>compose-up</code> · <code>compose-down</code> · <code>migrate</code> · <code>seed-contract-fixtures</code> | <code>npm ci</code> 와 <code>uv sync</code> 를 순서대로 묶거나, <code>DOCKER_CONTEXT</code> · <code>UV_CACHE_DIR</code> 환경변수를 주입해야 함 |
| 계약·경계 게이트 | <code>contracts-generate</code> · <code>contracts-check</code> · <code>boundaries-check</code> · <code>migration-check</code>             | 전용 파이썬 스크립트/셸 스크립트를 정해진 인터프리터 환경에서 실행                                                                                       |
| 회귀·부하     | <code>test-ai-regression</code> · <code>test-load-pilot</code>                                                                            | 러너가 백엔드 패키지를 직접 임포트하므로 <code>PYTHONPATH=backend/src:. 필요</code> — Makefile에 그 이유를 주석으로 남겨 뒀습니다                              |
| 인프라 검증    | <code>infra-format-check</code> · <code>infra-validate</code> · <code>infra-security-check</code> · <code>infra-plan-check</code>         | 4개 Terraform 루트를 순회하며 <code>init - backend=false</code> 후 <code>validate</code> — 셸 루프 필요                                   |
| 통합 검증     | <code>verify-foundation</code> · <code>test-local-production-parity</code>                                                                | 여러 게이트를 정해진 순서로 실행하는 셸 스크립트 위임                                                                                              |

## 함께 정리한 VERIFY\_FOUNDATION.SH

기존에는 스크립트가 Make 타겟을 부르고, 그 타겟이 다시 npm을 부르는 3단 간접 호출이었습니다. 지금은 `npm run format:check` → `lint` → `typecheck` → `pytest backend/tests/contract backend/tests/unit/shared` → 계약·경계·마이그레이션 게이트 3개 → `npm run build` 순서로 **직접** 실행합니다. 실패한 줄이 곧 실패한 명령입니다.

## 2.3 CI 구성 (변경 후)

```
.github/workflows/ci.yml  - 푸시·PR마다
  job: workspace
    - npm ci
    - uv sync --frozen --no-editable
    - npm run test:workspace  ← 포맷·린트·타입체크·전 테스트 한 번에
    - make contracts-check    ← 생성 계약이 정보와 일치하는가
    - make boundaries-check   ← 레인이 남의 private 경로를 임포트하지 않는가
    - make migration-check    ← 레인별 마이그레이션 헤드가 1개인가

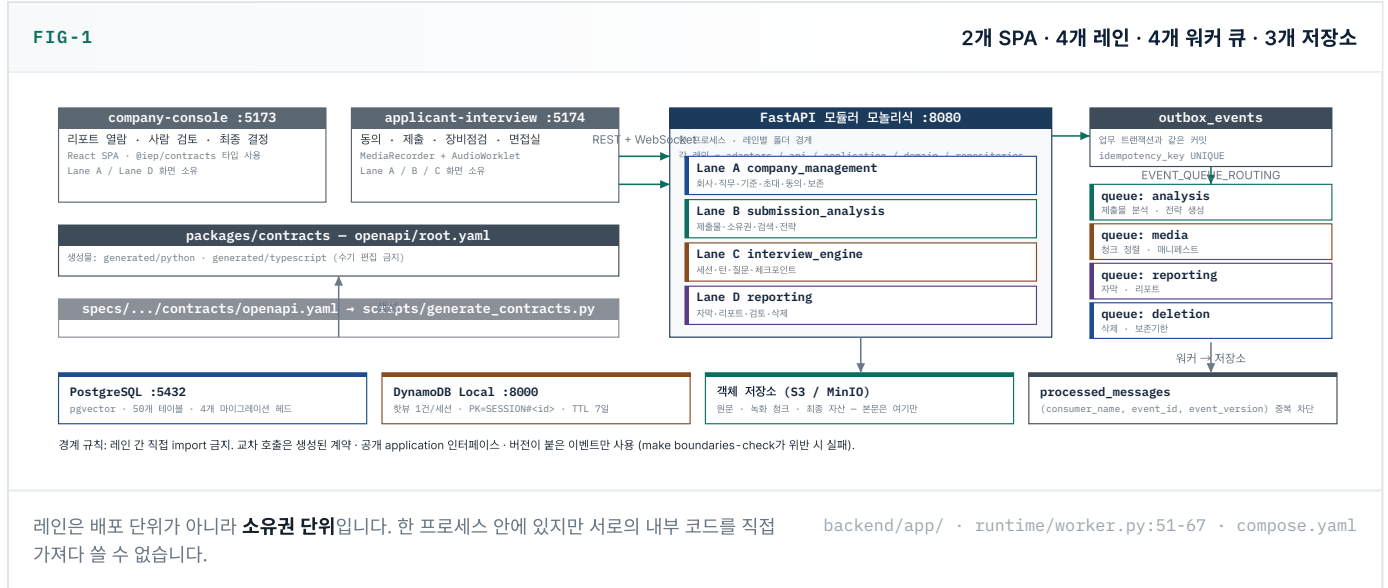
.github/workflows/e2e.yml  - 수동 실행 + 평일 야간(cron 0 18 * * 1-5)
  timeout 30분 · Playwright Chromium · make compose-up
  브라우저 여정은 Docker 기동이 필요해 매 푸시에 돌리기엔 과함 →
  야간 배치로 분리. 필요할 때 workflow_dispatch로 즉시 실행 가능.
```

### 확인

기준 커밋 `7c998be` 에서 `uv run --no-sync pytest backend/tests` 실행 결과 **193 passed, 1 skipped, 7.13초**. `git status --porcelain` 출력 0줄(클린).

## 03 시스템 토폴로지

화면 2개, 백엔드 1개 프로세스, 워커 큐 4개, 저장소 3종. 마이크로서비스가 아니라 **폴더로 경계를 나눈 하나의 프로세스**입니다.



### 3.1 왜 모놀리식인가 — 감리 판단

레이ンを 4개로 나눴으면서도 서비스를 4개로 쪼개지 않은 것은 **합리적인 선택**으로 봅니다. 근거는 실시간 경로입니다. 면접 1턴은 **음성 인식 → 검색 → 질문 생성 → 정책 검사 → 저장**을 지원자가 기다리는 동안 끝내야 하는데, 이 과정이 Lane B(검색)와 Lane C(면접)에 걸쳐 있습니다. 서비스가 분리돼 있었다면 이 구간에 네트워크 왕복이 최소 2회 추가됩니다. 대신 경계는 **make boundaries-check** 로 기계적으로 강제해 두었습니다 — 나중에 정말 쪼개야 할 때 쪼갤 수 있는 상태를 유지하는 구조입니다.

### 3.2 아웃박스가 하는 일

이벤트를 큐에 직접 넣지 않고 **업무 데이터와 같은 트랜잭션**으로 **outbox\_events** 테이블에 먼저 씁니다. 이 한 가지 선택이 두 종류의 사고를 동시에 막습니다.

| 사고 유형               | 아웃박스가 없다면                            | 아웃박스가 있으면                             |
|---------------------|--------------------------------------|---------------------------------------|
| 업무는 저장됐는데 이벤트 발행 실패 | 면접은 끝났는데 리포트가 영원히 생성되지 않음.<br>아무도 모름 | 같은 커밋이므로 불가능. 발행은 나중에 재시도             |
| 이벤트는 발행됐는데 업무 롤백    | 존재하지 않는 세션의 리포트를 만들려다 실패 반복          | 롤백되면 이벤트 행도 함께 사라짐                    |
| 같은 이벤트 중복 수신        | 리포트 2개, 자막 2배                        | <b>processed_messages</b> PK가 재처리를 차단 |

#### 중복 차단 키의 설계

**processed\_messages** 의 PK는 **(consumer\_name, event\_id, event\_version)** 입니다. **소비자 이름이 키에 들어 있다는 점이 중요합니다.** 같은 **interview.completed** 하나를 미디어 워커와 리포트 워커가 각각 한 번씩 처리해야 하는데, 키에 소비자가 없으면 먼저 처리한 쪽이 다른 쪽을 막아버립니다. 또 **event\_version** 이 키에 있어 이벤트 스키마가 v2로 올라가면 재처리가 가능합니다.

### 3.3 이벤트 → 큐 라우팅 전수

**runtime/worker.py:51-67** 의 **EVENT\_QUEUE\_ROUTING** 딕셔너리에 등록된 **15개 전부**입니다 (analysis 5 · reporting 5 · media 1 · deletion 4). 등록되지 않은 이벤트는 아웃박스에 남지만 어느 워커도 가져가지 않습니다.



| 큐         | 이벤트                                                                                                                                 |
|-----------|-------------------------------------------------------------------------------------------------------------------------------------|
| ANALYSIS  | system.parity_probe · invitation.consent_completed · submission.analysis_requested · submission.analysis_completed · strategy.ready |
| MEDIA     | media.postprocess_requested                                                                                                         |
| REPORTING | interview.turn_finalized · interview.session_paused · interview.completed · report.generation_requested · report.ready              |
| DELETION  | deletion.requested · deletion.target_requested · deletion.target_verified · retention.expired                                       |
| 미등록       | interview.checkpoint_changed ← F-05. application/checkpoints.py:57 에서 발행됨                                                           |

## 04 ERD — 전체 50개 테이블

4개 레인의 `repositories/postgres.py` 와 `shared/persistence.py` 를 전수 열람해 재구성했습니다. 추정한 컬럼은 없습니다.

|                                        |                                         |                                         |                                               |
|----------------------------------------|-----------------------------------------|-----------------------------------------|-----------------------------------------------|
| LANE C 면접<br><b>9</b><br>세션·턴·근거·체크포인트 | LANE B 분석<br><b>12</b><br>제출물·소유권·검색·전략 | LANE D 리포트<br><b>10</b><br>자막·리포트·검토·삭제 | LANE A + 공통<br><b>12+7</b><br>회사·초대 / 아웃박스·감사 |
|----------------------------------------|-----------------------------------------|-----------------------------------------|-----------------------------------------------|

### 4.1 50개 테이블 전부에서 지켜지는 3가지 규칙

| 규칙                                     | 구현                                                                                                                                                            | 감리 확인                                                                                                                                                                                            |
|----------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 업무 테이블 전부에서<br><code>company_id</code> | 단일 PK 테이블도 <code>company_id</code> 컬럼을 별도로 가짐.<br>조회 SQL의 <code>WHERE</code> 에 항상 포함                                                                          | <b>49/50 보유.</b> 유일한 예외는 <code>processed_messages</code> 로, 회사와 무관한 <code>consumer_name</code> 기록이며 PK가 <code>(consumer_name, event_id, event_version)</code> 입니다 — 업무 데이터를 담지 않으므로 테넌시 대상이 아닙니다 |
| 본문은 객체 저장소에                            | 제출물 원문은 <code>submission_chunks.text_object_key</code> (키만), 영상은 <code>recording_chunks.object_key</code>                                                     | 관계형 DB에 원문을 담는 컬럼은 <code>retrieval_documents.protected_text</code> 와 턴/자막 텍스트뿐 — 검색·근거에 필요한 최소 범위                                                                                                |
| 버전 컬럼이 데이터에 붙음                         | <code>embedding_model · embedding_version · extraction_version · generation_version · retrieval_config_version · model_config_version · prompt_version</code> | 모델을 교체해도 과거 판단이 어느 버전으로 만들어졌는지 남습니다. 재현 가능성의 전제                                                                                                                                                  |

### 4.2 낙관적 동시성 — `row_version` 이 있는 6개 테이블

전체 50개 중 `row_version` 컬럼을 가진 것은 6개입니다. 다만 컬럼이 있다는 것과 실제로 검사한다는 것은 다릅니다 — 확인 결과 3단계로 갈렸습니다.

| 테이블                       | 검사 지점                                              | 예외                                      | 감리 확인                                                                                                 |
|---------------------------|----------------------------------------------------|-----------------------------------------|-------------------------------------------------------------------------------------------------------|
| positions                 | API 헤더<br><code>If-Match-Version</code>            | <code>StalePositionVersionError</code>  | 완전 적용. <b>PATCH /POSITIONS/{POSITION_ID}</b> 가 헤더를 필수로 받고 <b>POSITION.REVISE()</b> 가 비교               |
| competency_model_versions | API 헤더<br><code>If-Match-Version</code>            | <code>StaleCriterionVersionError</code> | 완전 적용. <b>POST .../PUBLISH</b> 가 헤더 필수. 발행은 한 번만 성공해야 하므로 여기에 반드시 필요                                  |
| invitations               | 도메인 내부<br><code>transition_to()</code>             | <code>InvitationStateError</code>       | 도메인에서 검사하지만 클라이언트가 헤더로 버전을 넘기지 않습니다. 호출부가 방금 읽은 <b>ROW_VERSION</b> 을 그대로 넘기므로 동시 수정을 실제로 잡지는 못합니다     |
| submissions               | 도메인 상태 전이 + 아웃박스<br><code>aggregate_version</code> | —                                       | 전이 시 <b>ROW_VERSION + 1</b> 이 되고 그 값이 이벤트의 <b>AGGREGATE_VERSION</b> 으로 나갑니다. 동시성 방어보다 이벤트 순서 식별에 쓰입니다 |
| interview_sessions        | 상태 전이 시 증가만                                        | —                                       | 비교하는 코드가 없습니다. 세션은 <b>SESSION_SEQUENCE</b> 로 순서를 통제하고 <b>ROW_VERSION</b> 은 증가만 합니다                    |
| interviewer_profiles      | —                                                  | —                                       | 컬럼만 있고 증가·비교 코드가 모두 없습니다. 사용되지 않는 컬럼                                                                  |

**절제로 보는 부분:** `interview_turns` 는 append-only이고 (초안 → FINAL 한 방향), `evidence · report_items` 는 워커가 한 번만 만듭니다. 이런 테이블에 버전 컬럼을 붙이면 보호 없이 비용만 늘어납니다. 44개 테이블에 안 붙인 것은 맞는 판단입니다.

`interview_sessions` 가 `session_sequence` 로 순서를 통제하는 것도 합리적입니다 — 웹소켓 메시지 순서는 버전 비교가 아니라 시퀀스로 다루는 편이 자연스럽습니다.

**미완으로 보는 부분:** `interviewer_profiles.row_version` 은 아무도 읽지 않고 아무도 늘리지 않습니다. 그리고 `invitations` 는 도메인이 버전을 검사하지만 호출부가 방금 읽은 값을 그대로 넘기므로 **동시 수정 방어가 실질적으로 작동하지 않습니다**. 초대 상태를 두 담당자가 동시에 바꾸는 상황이 실제로 문제가 되는지 판단하고, 필요하다면 `positions` 처럼 헤더를 필수로 만들거나 필요 없으면 컬럼을 제거하는 것이 좋습니다. 지금은 *보호되는 것처럼 보이지만 보호되지 않는 상태*입니다.

### 4.3 사람 검토가 AI 원본을 덮어쓰지 않는 구조

이 부분은 별도로 확인했습니다. `reports` 에는 `kind` 컬럼이 있고 워커는 항상 `AI_ORIGINAL` 로 씁니다. 사람의 수정은 `reports` 를 UPDATE 하지 않고 `human_reviews` 에 **새 행을 추가**합니다( `review_type` , `target_id` , `value` JSON, `reason` , `company_user_id` ).

#### 감리 의견

"AI가 이렇게 판단했고, 사람이 이렇게 뒤집었다"가 **둘 다 남습니다**. 채용 분쟁에서 가장 필요한 기록 형태입니다. 만약 사람이 리포트를 직접 수정하는 구조였다면 AI가 원래 무엇을 말했는지 복원할 방법이 없습니다. 다음 ERD 5장에서 이 관계를 확인할 수 있습니다.

## 4.4 LANE C 실시간 면접 — 9개 테이블

면접 중 1턴이 진행될 때 어떤 행이 어떤 순서로 생기는지 이 ERD에 담겨 있습니다. 가장 중요한 관계는 `interview_turns` 하 나에 `question_rationales` (왜 물었나)와 `question_source_references` (무엇을 근거로)가 붙는 부분입니다.

● PK · 굵게

● FK · 파란 글씨

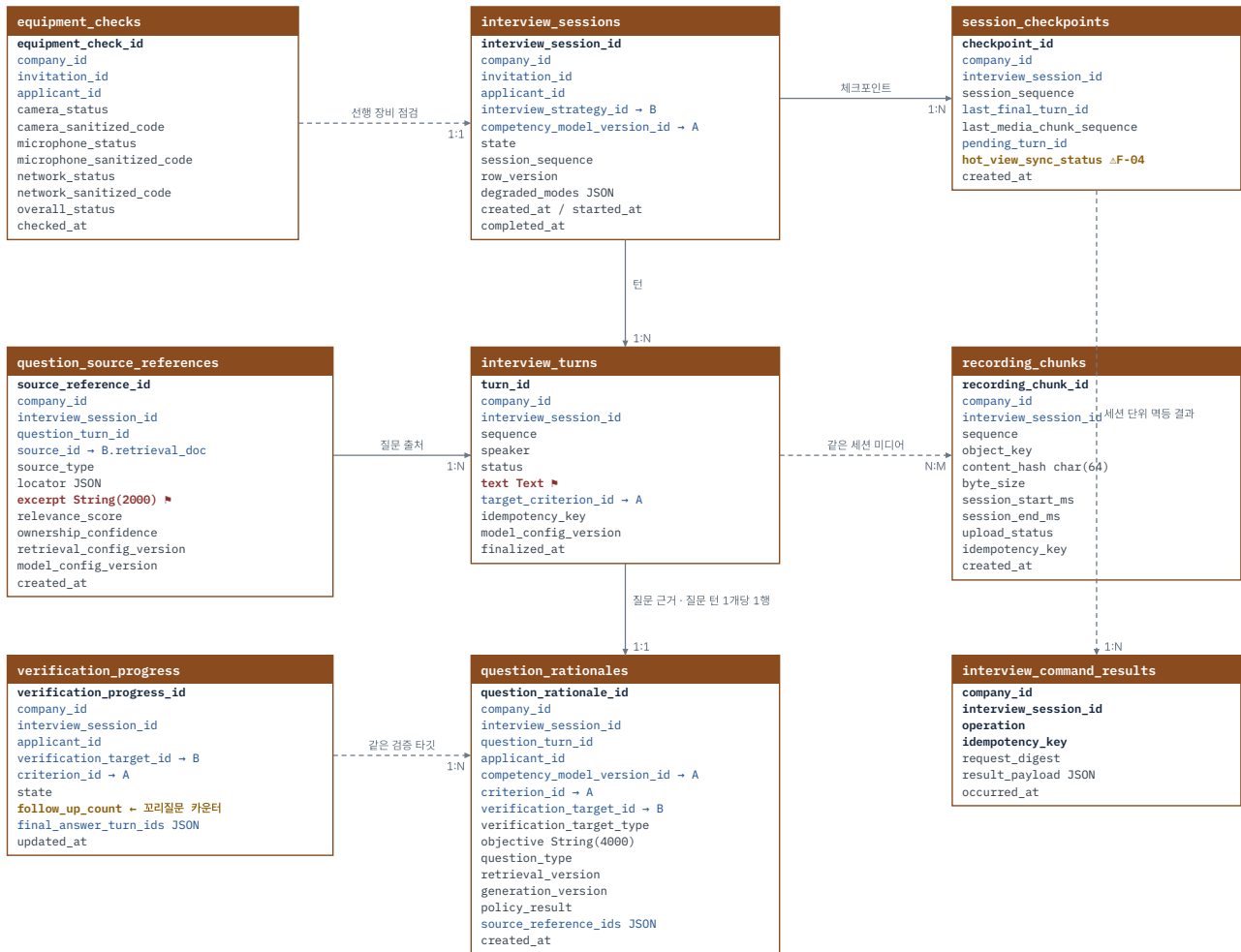
● 보호 대상 텍스트

● △ 감리 지적 컬럼

실선 = 외래 키 · 점선 = JSON 컬럼을 통한 느슨한 참조

FIG-2

Lane C · interview\_engine — 세션 · 턴 · 근거 · 체크포인트



`interview_command_results`는 유일하게 복합 PK (`company_id`, `interview_session_id`, `operation`, `idempotency_key`) 를 씁니다 — 멍등성을 인덱스가 아니라 키 제약으로 강제합니다.

backend/src/interview\_engine/repositories/postgres.py · application/idempotency.py:97

| 테이블                        | 이 테이블이 없으면 무엇이 불가능해지는가                                                                                          |
|----------------------------|-----------------------------------------------------------------------------------------------------------------|
| equipment_checks           | 면접 시작 전 카메라·마이크·네트워크 상태를 남기지 못함 → 나중에 "장비 문제로 답을 못 했다"는 이의를 확인할 근거가 없음. *_sanitized_code 로 원문 오류 메시지가 아닌 코드만 저장 |
| interview_sessions         | 서버 권위 상실. state · session_sequence · row_version · degraded_modes 가 모두 여기 있음                                    |
| interview_turns            | 대화 자체가 사라짐. speaker 로 질문/답변 구분, status 로 초안/확정 구분                                                               |
| session_checkpoints        | 재접속 복구 불가. 마지막 확정 턴과 마지막 청크 번호를 알 수 없음                                                                          |
| recording_chunks           | 녹화 무결성 증명 불가. content_hash 는 64자 고정 검사                                                                          |
| question_source_references | "이 질문의 근거는 제출물 어디인가"를 답할 수 없음 → 근거 없는 질문과 구별 불가                                                                 |
| verification_progress      | 꼬리질문 판단 불가. follow_up_count 가 유일한 카운터                                                                           |
| question_rationales        | "왜 이 질문을 했는가"의 감사 기록 상실. policy_result 로 정책 통과/거부까지 남음                                                          |
| interview_command_results  | 중복 명령 방어 상실 → 같은 답변으로 턴 2개 생성                                                                                   |

## 4.5 LANE B 제출물 분석과 검색 — 12개 테이블

면접 전에 돌아가는 구간입니다. 제출물이 들어와 **검증 타킷**(무엇을 확인해야 하는가)으로 바뀌는 전 과정이 여기 있습니다.

● PK · 굵게

● FK · 파란 글씨

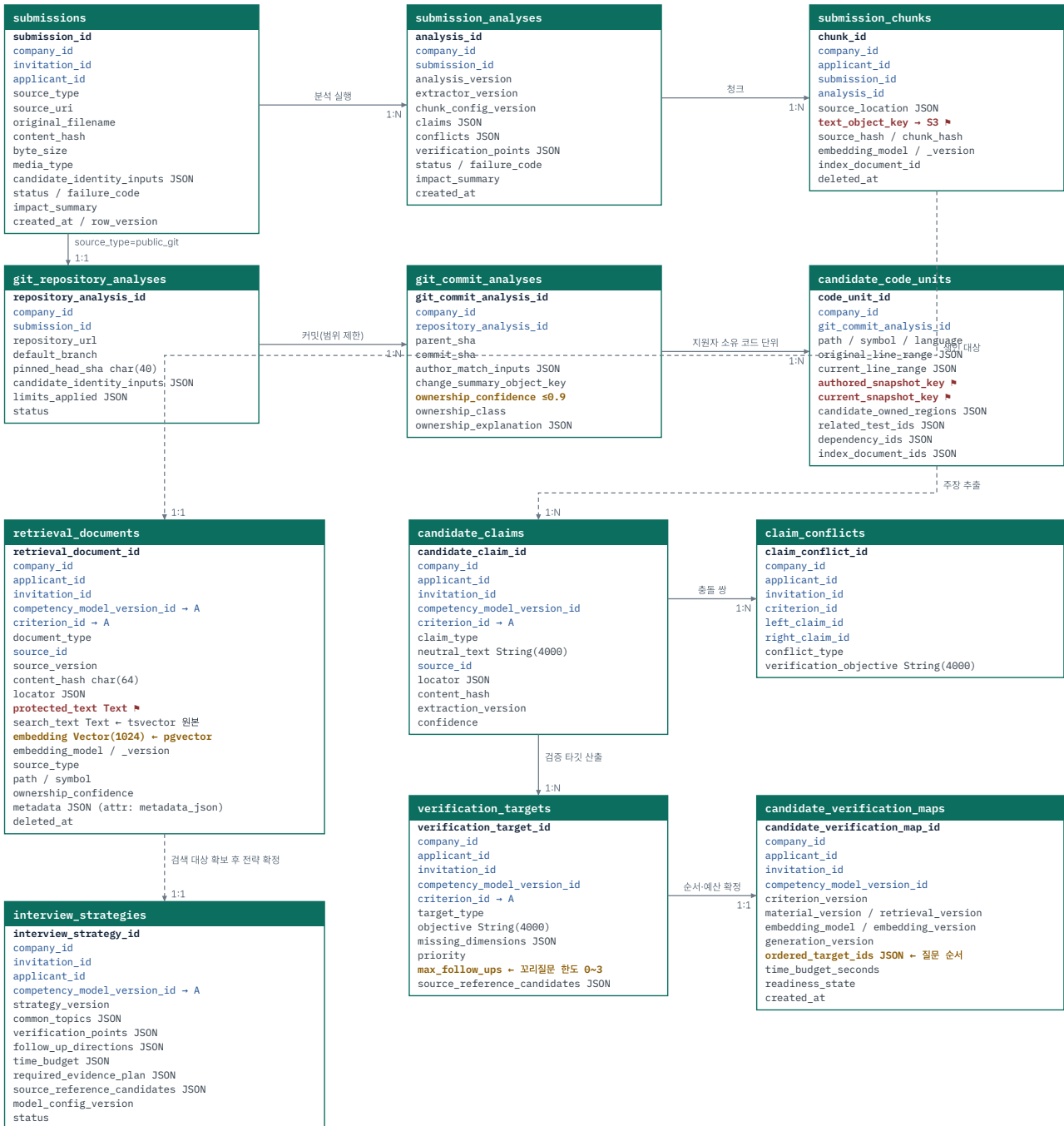
● 보호 대상 텍스트

● △ 감리 지적 컬럼

실선 = 외래 키 · 점선 = JSON 컬럼을 통한 느슨한 참조

FIG-3

Lane B · submission\_analysis — 제출물 · 소유권 · 검색 · 전략



**retrieval\_documents**가 이 레인의 심장입니다. **protected\_text** (원문)와 **search\_text** (검색용)를 분리해 두고, **embedding** 은 **Vector(1024)** 입니다. **metadata\_json** 속성이 실제 컬럼명 **metadata** 에 매핑됩니다.

backend/src/submission\_analysis/repositories/postgres.py

### 소유권 판정 3단 테이블

공개 Git 저장소를 제출받았을 때 "이 코드를 이 지원자가 썼는가"를 판정하는 경로가 **git\_repository\_analyses** → **git\_commit\_analyses** → **candidate\_code\_units** 3단입니다.

| 테이블                     | 핵심 컬럼                                                                   | 감리 관점                                                                                                          |
|-------------------------|-------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------|
| git_repository_analyses | pinned_head_sha(40) · limits_applied JSON                               | 커밋 SHA를 <b>고정</b> 합니다. 지원자가 나중에 저장소를 수정해도 면접에 쓴 시점의 코드가 특정됩니다.<br><code>limits_applied</code> 는 무한 클론 방지 상한 기록 |
| git_commit_analyses     | ownership_confidence · ownership_class · ownership_explanation JSON     | 점수만 남기지 않고 <b>왜 그 점수인지를</b> JSON으로 남깁니다 ( <code>no_identity_match</code> 같은 코드)                                |
| candidate_code_units    | original_line_range / current_line_range · candidate_owned_regions JSON | 파일 단위가 아니라 <b>줄 범위 단위</b> 로 소유권을 봅니다. "이 파일 중 이 함수만 본인이 썼다"를 표현 가능                                             |

소유권 점수 상한이 0.9인 이유

`domain/git_analysis.py:51` 의 계산식은 `min(0.9, 이름 0.35 + 이메일 0.45 + 핸들 0.2)` 입니다. 세 신호가 모두 맞아도 1.0이 되지 않습니다. 커밋 작성자 정보는 위조가 가능하므로 어떤 조합으로도 "확실"이라고 말하지 않겠다는 선언입니다. 그리고 0.5 미만이면 주장으로 쓰지 않고 `OWNERSHIP_UNCERTAIN` 검증 타깃으로 바꿔 *면접에서 직접 물어봅니다*. 애매한 것을 점수로 뭉개지 않고 질문으로 넘기는 설계입니다.

## 4.6 LANE D 근거 · 리포트 · 검토 · 삭제 — 10개 테이블

면접이 끝난 뒤의 구간입니다. 발견 사항 F-01·F-02·F-03이 모두 이 레인에 몰려 있습니다.

● PK · 굵게

● FK · 파란 글씨

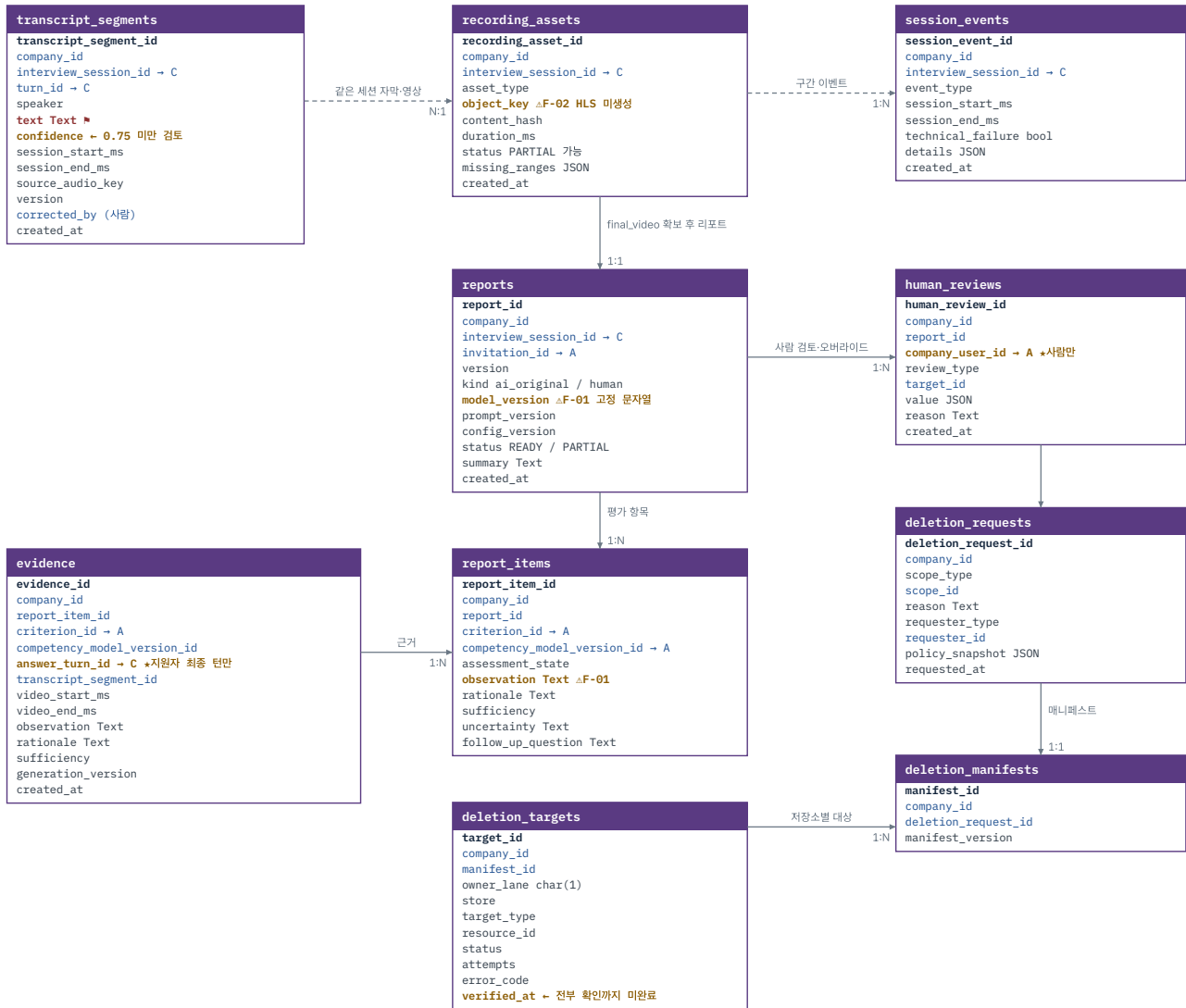
● 보호 대상 텍스트

● △ 감리 지적 컬럼

실선 = 외래 키 · 점선 = JSON 컬럼을 통한 느슨한 참조

FIG-4

Lane D · reporting — 자막 · 리포트 · 사람 검토 · 삭제 검증



evidence가 answer\_turn\_id · transcript\_segment\_id · video\_start\_ms/end\_ms 를 동시에 요구하는 것이 이 시스템의 핵심 불변식입니다.

backend/src/reporting/repositories/postgres.py

### 삭제가 3개 테이블로 나뉜 이유

삭제 요청 하나가 deletion\_requests → deletion\_manifests → deletion\_targets 로 펼쳐집니다. 관계형 DB 한 곳만 지우면 되는 일이 아니기 때문입니다.



| 테이블                | 핵심 컬럼                                                                        | 역할                                                                                                                             |
|--------------------|------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------|
| deletion_requests  | scope_type · scope_id · requester_type · requester_id · policy_snapshot JSON | "누가 · 무엇을 · 어떤 정책으로" 요청했는지. <code>policy_snapshot</code> 은 <b>요청 당시 보존 정책을 복사해</b> 둡니다 — 정책이 나중에 바뀌어도 이 삭제가 어떤 규칙으로 실행됐는지 남습니다 |
| deletion_manifests | manifest_id · manifest_version                                               | 삭제 대상 목록의 버전. 목록 산출 로직이 개선되면 버전이 올라갑니다                                                                                         |
| deletion_targets   | owner_lane char(1) · store · status · attempts · error_code · verified_at    | 저장소별 개별 항목. <code>owner_lane</code> 이 <b>1글자(A/B/C/D)</b> 로 어느 레인 책임인지 표시                                                      |

**VERIFIED\_AT** 이 있고 **DELETED\_AT** 이 없는 이유

삭제 항목의 완료 조건이 "지웠다"가 아니라 **"없음을 확인했다"**입니다. 지우기 API를 호출해 200을 받은 것과, 실제로 조회해서 없는 것은 다릅니다. 특히 검색 인덱스와 DynamoDB 핫뷰는 삭제가 즉시 반영되지 않을 수 있습니다. `attempts` 와 `error_code` 가 함께 있는 것은 **재시도를 전제한 설계**이며, 모든 `deletion_targets` 가 `verified_at` 을 갖기 전까지 삭제 요청은 미완료로 남습니다. 감리에서 잘 설계된 것으로 판단합니다.

이 레인에서 확인된 미완성 3건

- **F-01** `report_items.observation` 이 "지원자의 최종 답변에서 관찰된 내용" 고정 문자열. `model_version` 도 "bedrock-model-v1" 상수 — 모델 호출 자체가 없습니다
- **F-02** `recording_assets.object_key` 에 `.../final/v1/manifest.m3u8` 가 기록되지만 HLS 변환 작업은 생성되지 않습니다
- **F-07** 자막 `confidence` 가 `0.9` 고정 — 실제 음성 인식 신뢰도가 근거 검증에 반영되지 않습니다

## 4.7 LANE A 회사 · 직무 · 기준 · 초대 · 동의 — 12개 테이블

모든 것의 출발점입니다. **평가 기준(competency model)**이 여기서 만들어지고 발행되며, 그 버전 ID가 이후 모든 레인으로 전파됩니다.

● PK · 굵게

● FK · 파란 글씨

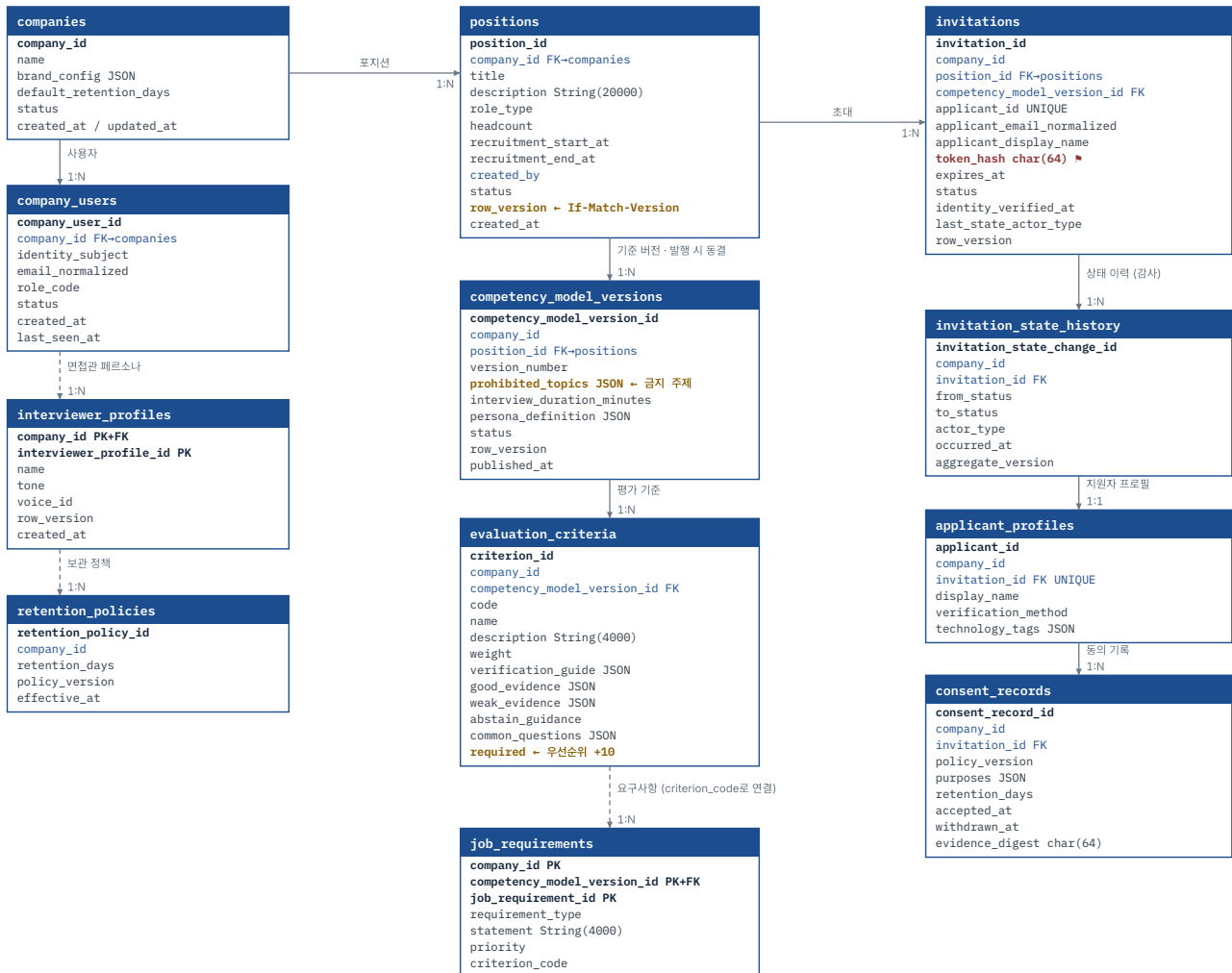
● 보호 대상 텍스트

● △ 감리 지적 컬럼

실선 = 외래 키 · 점선 = JSON 컬럼을 통한 느슨한 참조

FIG-5

Lane A · company\_management — 테넌트 · 기준 버전 · 초대 · 동의 · 보존



invitations.token\_hash는 64자 해시입니다 — 초대 토큰 원문은 DB에 저장되지 않습니다.

backend/src/company\_management/repositories/postgres.py

applicant\_id 가 UNIQUE라 한 초대에 한 지원자만 연결됩니다.

| 테이블 / 제약                                     | 감리 관점에서 중요한 이유                                                                                                                           |
|----------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------|
| competency_model_versions                    | 기준을 <b>버전으로</b> 관리합니다. 발행( <code>PUBLISHED</code> )된 버전만 초대에 붙을 수 있어, 작성 중인 기준으로 면접이 시작되지 않습니다                                           |
| evaluation_criteria                          | 기준 버전 하나에 여러 평가 항목. 리포트의 <code>report_items</code> 가 1:1로 대응하는 대상                                                                        |
| job_requirements<br>(3중 복합 PK)               | <code>(company_id, competency_model_version_id, job_requirement_id)</code> —<br>요건이 <b>기준 버전에 종속됨</b> 을 키로 강제. 버전을 넘나드는 요건 참조가 구조적으로 불가능 |
| interviewer_profiles<br>(복합 PK)              | <code>(company_id, interviewer_profile_id)</code> — 회사 없는 면접관 프로필이 존재할 수 없음                                                              |
| invitations.token_hash(64)                   | 초대 링크의 토큰을 <b>해시로만</b> 보관. DB가 유출돼도 초대 링크를 복원할 수 없습니다                                                                                    |
| invitation_state_history                     | 초대 상태 변화를 <b>덮어쓰지 않고 누적</b> . "언제 발송됐고 언제 열렸는가"를 재구성 가능                                                                                  |
| applicant_profiles<br>(invitation_id UNIQUE) | 초대와 지원자 프로필이 1:1. 한 초대 링크를 여러 사람이 쓸 수 없습니다                                                                                               |
| consent_records                              | 동의를 별도 테이블로 분리. 어떤 목적(purpose)에 언제 동의했는지가 남습니다 — 개인정보 처리의 법적 근거                                                                          |
| retention_policies                           | 보존 기한 정책. <code>retention.expired</code> 이벤트가 deletion 큐로 들어가는 출발점                                                                       |

감리 의견 — 초대 설계

초대 하나가 **기준 버전 · 지원자 · 동의 · 제출물 · 면접 · 리포트 전부의 범위 키**가 됩니다. `invitation_id` 만 있으면 그 채용 회차의 모든 데이터를 정확히 특정할 수 있고, 반대로 `invitation_id` 가 다르면 어떤 조회로도 섞이지 않습니다. 삭제 요청의 `scope_type / scope_id` 가 이 구조에 의존합니다 — 지원자 단위 삭제가 깔끔하게 성립하는 이유입니다.

## 4.8 공통 인프라 테이블 — 7개

어느 레인도 소유하지 않고 **Integration**이 소유하는 테이블입니다. 이벤트 전달, 감사 로그, 지원자 접근 토큰이 여기 있습니다.

● PK · 굵게

● FK · 파란 글씨

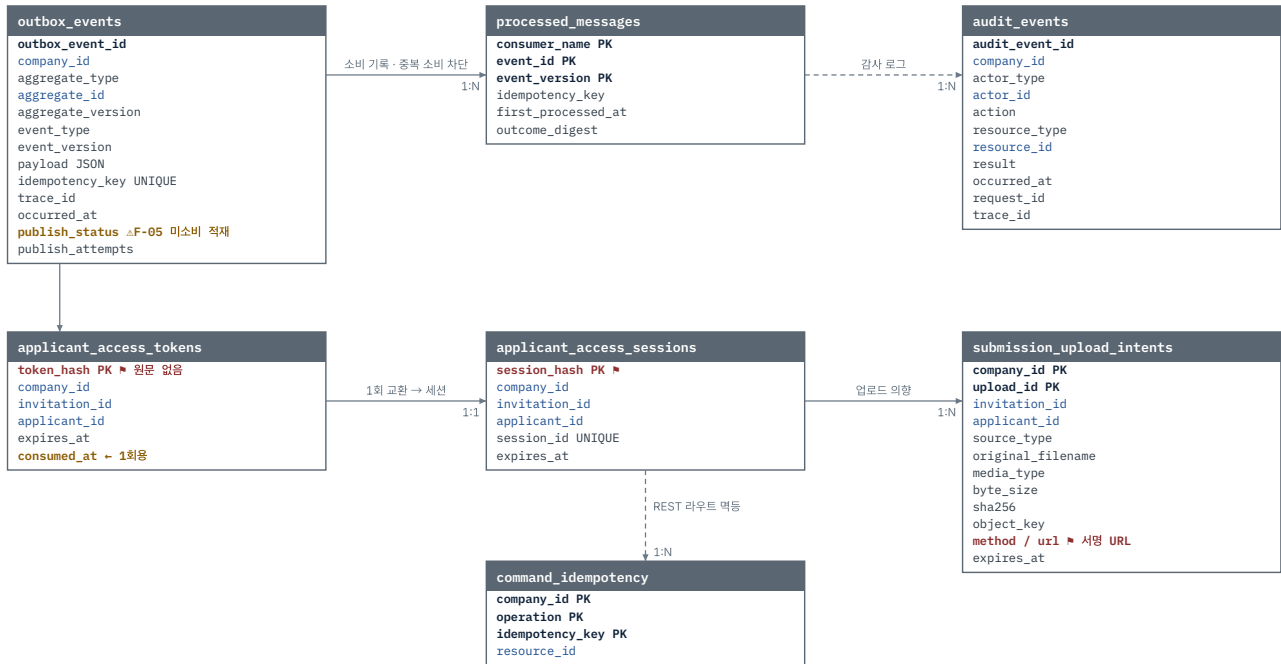
● 보호 대상 텍스트

● △ 감리 지적 컬럼

실선 = 외래 키 · 점선 = JSON 컬럼을 통한 느슨한 참조

FIG-6

shared/persistence.py — 아웃박스 · 중복 차단 · 감사 · 접근 토큰 · 업로드 의도



접근 토큰 계열 테이블은 모두 해시를 PK로 씁니다 — `applicant_access_tokens.token_hash`, `applicant_access_sessions.session_hash`. 원문 토큰은 어디에도 저장되지 않습니다.

backend/src/shared/persistence.py

| 테이블                       | 키 설계                                                       | 왜 그렇게 했는가                                                                       |
|---------------------------|------------------------------------------------------------|---------------------------------------------------------------------------------|
| outbox_events             | idempotency_key UNIQUE<br>+ aggregate_version · trace_id   | 같은 업무 동작이 두 번 실행돼도 이벤트는 하나. <code>trace_id</code> 로 "이 리포트는 어느 요청에서 시작됐나"를 추적   |
| processed_messages        | PK (consumer_name, event_id, event_version)                | 소비자별 독립 중복 차단. 3.2절 설명 참조                                                       |
| audit_events              | actor_type · actor_id · action<br>· resource_type · result | 행위자 유형이 컬럼으로 분리돼 있어 "AI가 한 것"과 "사람이 한 것"을 구별합니다. 최종 결정에 AI가 개입하지 않았음을 이 테이블로 증명 |
| applicant_access_tokens   | PK token_hash<br>+ consumed_at                             | <code>consumed_at</code> 이 있어 일회용임을 강제. 링크를 두 번 쓸 수 없습니다                        |
| applicant_access_sessions | PK session_hash<br>+ session_id UNIQUE · expires_at        | 토큰 교환 후의 세션도 해시로만 보관. 만료 시각 필수                                                  |
| submission_upload_intents | PK (company_id, upload_id)<br>+ sha256 · expires_at        | 업로드 전에 해시를 선언받습니다. 업로드 후 실제 해시와 비교해 다르면 거부 — 서명 URL 재사용 공격 차단                   |
| command_idempotency       | PK (company_id, operation, idempotency_key) → resource_id  | REST 명령용. 결과 전체가 아니라 <code>resource_id</code> 만 저장 — 재요청 시 같은 자원을 가리키게만 하면 충분   |

README에 "*applicant source text, answer text, production credentials, raw invitation tokens or signed URLs*를 설정이나 로 그에 두지 말라"는 규칙이 있고, ERD가 그 규칙과 **일치**합니다. 원문 토큰을 저장하는 컬럼이 하나도 없고, 오류 응답 봉투에는 `request_id` 만 들어갑니다. 선언과 구현이 어긋나지 않은 드문 경우입니다.

## 05 ID 계보

ERD를 다 봐도 놓치기 쉬운 것이 **같은 ID가 여러 테이블을 관통한다는 사실**입니다. 이 장은 그 관통 경로만 따로 봅니다.

FIG-7

기준 버전 고정과 지원자 범위 잠금



competency\_model\_version\_id는 이 다섯 테이블에 모두 같은 값으로 저장됩니다.  
면접이 진행되는 중에 회사가 기준을 수정해도, 이미 시작된 면접과 그 리포트는 면접 당시 버전으로 평가됩니다. — “평가 기준이 바뀌었다”는 의미 제기를 데이터로 반박할 수 있습니다.

지원자 범위 잠금 — 모든 조회에 3중 조건

|                                                         |                                         |                                  |                                                              |
|---------------------------------------------------------|-----------------------------------------|----------------------------------|--------------------------------------------------------------|
| <b>company_id</b><br>테넌트 검색 · 리포지토리 SQL의 WHERE 절에 항상 포함 | <b>applicant_id</b><br>지원자 범위 · 초대와 1:1 | <b>invitation_id</b><br>제출 회차 범위 | <b>criterion_id / verification_target_id</b><br>평가 속 · 검증 단위 |
|---------------------------------------------------------|-----------------------------------------|----------------------------------|--------------------------------------------------------------|

다른 회사의 ID로 조회하면 403이 아니라 404입니다.  
403은 “그 자원은 있지만 권한이 없다”를 알려주는 자원의 존재 자체가 정보가 됩니다. 404는 존재 여부조차 노출하지 않습니다. — 이 결정은 리포지토리 SQL에서 강제되며 서비스 측 실수로 우회되지 않습니다.

이 두 가지가 지켜지지 않으면 다른 모든 안전장치가 무의미해집니다. 기준 버전이 흔들리면 평가의 일각 레인 repositories/postgres.py · shared/tenancy.py  
관성이 사라지고, 범위 잠금이 깨지면 테넌트 간 데이터가 섞입니다.

### 5.1 기준 버전이 고정되는 지점 4곳

| 단계     | 저장 위치                                                                   | 고정되는 순간                                      |
|--------|-------------------------------------------------------------------------|----------------------------------------------|
| 초대 생성  | invitations.competency_model_version_id                                 | 담당자가 초대를 만들 때. 이후 회사가 기준을 수정해도 이 초대에는 바뀌지 않음 |
| 전략 생성  | interview_strategies.competency_model_version_id                        | 제출물 분석이 끝날 때. 초대의 값을 그대로 복사                  |
| 세션 시작  | interview_sessions.competency_model_version_id                          | 면접 시작 시. 전략의 값을 그대로 복사                       |
| 리포트 생성 | report_items · evidence<br>· verification_targets ·<br>candidate_claims | 리포트 워커가 항목을 만들 때. 세션의 값을 그대로 복사              |

#### 이것이 실무에서 왜 중요한가

채용 분쟁에서 가장 흔한 주장이 “평가 기준이 도중에 바뀌었다”입니다. 이 구조에서는 그 주장을 데이터로 반박할 수 있습니다 — 초대·전략·세션·리포트 네 곳에 같은 버전 ID가 박혀 있으므로, 면접 당시 어떤 기준이 적용됐는지가 확정됩니다.

반대 방향도 성립합니다. 회사가 기준을 개선하려 할 때 **진행 중인 면접에 영향을 줄까 걱정할 필요가 없습니다**. 새 버전은 새 초대부터 적용됩니다. quickstart 문서의 종단 검증 항목에도 “the criterion version is identical across invitation, strategy, session and report”가 명시되어 있고, 코드가 그것을 지킵니다.

### 5.2 왜 403이 아니라 404인가

다른 회사의 자원을 조회하면 **404**가 반환됩니다. 감리에서 이 선택을 확인하고 **맞는 결정**으로 판단했습니다.

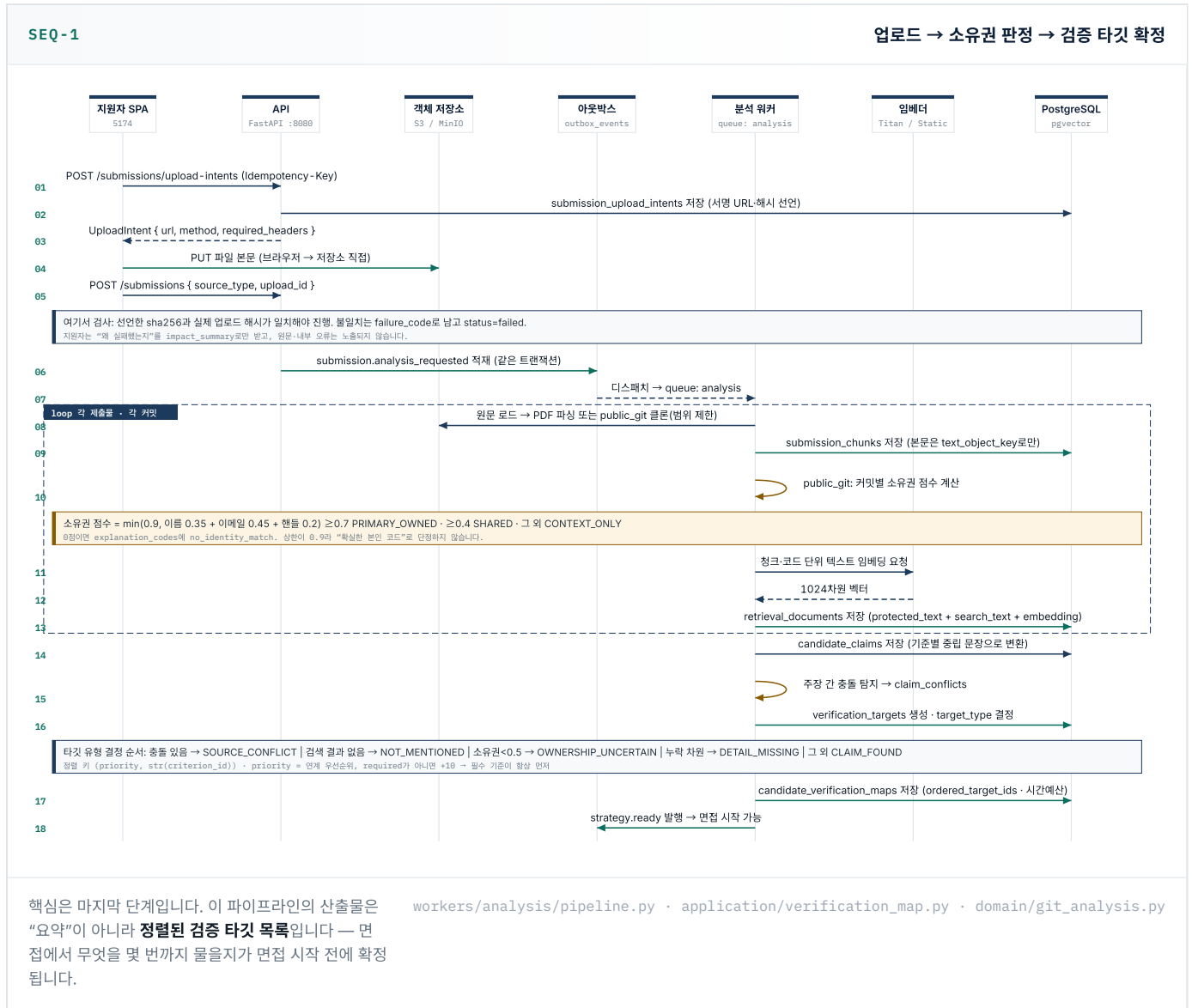
| 응답         | 공격자가 알게 되는 것                  | 실제 위험                                                                  |
|------------|-------------------------------|------------------------------------------------------------------------|
| <b>403</b> | “그 ID의 자원은 존재한다. 다만 내 권한이 없다” | ID를 순회하며 403/404를 구분하면 다른 회사에 어떤 자원이 있는지 목록을 만들 수 있습니다. ID 자체가 정보가 됩니다 |
| <b>404</b> | 존재 여부를 알 수 없음                 | 순회해도 아무 정보를 얻지 못함                                                      |

구현 위치가 더 중요합니다

이 404는 API 계층의 `if` 문이 아니라 리포지토리 SQL의 `WHERE company_id` 에서 나옵니다. 조건에 맞는 행이 없으니 `None` 이 반환되고, 그것이 404가 됩니다. 서비스 계층에서 권한 검사를 빠뜨려도 데이터가 새지 않는다는 뜻입니다. 권한 검사를 API에만 두는 구조는 새 엔드포인트를 추가할 때마다 실수 가능성이 생기지만, 여기서는 쿼리 자체가 막힙니다.

## 06 시퀀스 1 — 면접 전 제출물 분석

지원자가 이력서나 GitHub 링크를 내면, 면접 시작 전에 "무엇을 확인해야 하는가"의 목록이 완성됩니다.



### 6.1 업로드가 2단계인 이유

파일을 API로 바로 올리지 않고 `upload-intents` 로 의도를 먼저 등록한 뒤 브라우저가 객체 저장소로 직접 올립니다. 이유가 세 가지입니다.

- **API 프로세스가 파일 본문을 만지지 않습니다.** 100MB 이력서가 실시간 면접 프로세스의 메모리를 잡아먹는 사고가 구조적으로 불가능합니다.
- **해시를 미리 선언받습니다.** `submission_upload_intents.sha256` 에 선언값이 저장되고, 업로드 후 실제 해시와 비교합니다. 다르면 `status=failed` 가 되고 `failure_code` 가 남습니다.
- **서명 URL에 만료가 있습니다.** `expires_at` 이 지나면 그 URL로는 올릴 수 없습니다.

### 6.2 이 단계에서 확정되는 것 — 꼬리질문 한도

감리에서 가장 눈여겨본 부분입니다. 꼬리질문을 몇 번까지 할지가 면접 전에 정수로 확정됩니다.



**verification\_targets** (분석 위커가 면접 전에 생성)

- | verification\_target\_id
- | criterion\_id ← 어느 평가 기준을 위한 타겟인가
- | target\_type ← 왜 확인이 필요한가 (5종)
- | objective (4000자) ← 무엇을 확인해야 하는가
- | priority ← 몇 번째로 물을 것인가
- | max\_follow\_ups ← 꼬리질문 상한. 면접 중에 바뀌지 않음

**candidate\_verification\_maps** (타겟들의 순서와 예산)

- | ordered\_target\_ids JSON ← 정렬된 타겟 ID 목록
- | time\_budget\_seconds ← 전체 면접 시간 예산
- | readiness\_state ← 면접 시작 가능 여부

#### 감리 의견 — "AI가 알아서 물어본다"가 아닙니다

면접 중 AI의 자유도는 **질문 문장을 만드는 것**으로 제한됩니다. 무엇을 물을지( `objective` ), 몇 번 물을지( `max_follow_ups` ), 어떤 순서로 물을지( `ordered_target_ids` )는 면접 전에 결정됩니다. 이는 두 가지를 보장합니다 — ① 면접 시간이 예측 가능하고, ② 같은 기준으로 지원한 사람들이 **같은 범위의 검증**을 받습니다. 모델이 그날 기분에 따라 어떤 지원자에게는 5번 캐묻고 다른 지원자에게는 1번만 묻는 일이 발생하지 않습니다.

### 6.3 검증 타깃 5종 — 판정 순서가 곧 우선순위

`application/verification_map.py` 에서 타깃 유형을 결정하는 순서입니다. **먼저 걸리는 조건이 이깁니다.** 이 순서 자체가 설계 의도를 드러냅니다.

| 순 | 유형                  | 조건                                         | 면접에서 무엇을 묻게 되는가                                 |
|---|---------------------|--------------------------------------------|-------------------------------------------------|
| 1 | SOURCE_CONFLICT     | 같은 기준에 대해 서로 다른 주장이 발견됨                    | "이력서에는 A라고 쓰셨는데 저장소에는 B로 보입니다" — <b>모순이 최우선</b> |
| 2 | NOT_MENTIONED       | 해당 기준으로 검색해 결과가 0건                         | "이 기준에 해당하는 경험을 말씀해 주세요" — 자료가 없으니 직접 물음        |
| 3 | OWNERSHIP_UNCERTAIN | <code>ownership_confidence &lt; 0.5</code> | "이 코드의 어느 부분을 직접 작성하셨나요" — <b>추측하지 않고 확인</b>    |
| 4 | DETAIL_MISSING      | 주장은 있으나 필요한 차원이 빠짐                         | "규모는 어느 정도였나요" 같은 구체화 질문                        |
| 5 | CLAIM_FOUND         | 위 어느 것도 아님                                 | 주장을 근거로 심화 질문                                   |

이 순서가 말해주는 것

**1번이 모순, 3번이 소유권 불확실**이라는 배치가 이 시스템의 성격을 정의합니다. "지원자의 좋은 점을 찾는 도구"가 아니라 **"제출한 자료와 실제 역량의 차이를 확인하는 도구"**입니다.

특히 3번 `OWNERSHIP_UNCERTAIN` 은 중요한 설계입니다. 소유권 점수가 낮을 때 *"본인 코드가 아님"*으로 **판정하지 않고** 질문으로 넘깁니다. 자동 판정이 사람의 해명 기회를 빼앗지 않는 구조입니다. quickstart의 Lane B 검증 항목에도 *"show that low ownership confidence produces a verification question, not a claim"*으로 명시돼 있습니다.

### 6.4 정렬 규칙

```
정렬 키 = (priority, str(criterion_id))
├ priority      = 연계 우선순위. required가 아닌 기준은 +10
└ criterion_id = 문자열 정렬 (동일 priority 내에서 결정적 순서 보장)
```

결과

필수 기준의 타깃이 항상 앞에 옵니다.

면접 시간이 부족해 뒤쪽 타깃을 못 물어도, **못 물은 것은 선택 기준**입니다.

같은 입력이면 항상 같은 순서 — 재현 가능한 면접.

**감리 확인**

시간이 부족할 때 **무엇을 포기할지가 미리 결정**돼 있습니다. 이것이 없으면 "면접이 일찍 끝났는데 정작 필수 기준을 안 물었다"가 발생합니다. 또 정렬 키에 `str(criterion_id)` 가 들어가 **동점일 때도 순서가 결정적**입니다 — 같은 지원자를 두 번 분석하면 같은 순서가 나옵니다. AI 회귀 테스트가 가능한 전제 조건입니다.

## 07 하이브리드 검색(RAG)

"레드 검색"으로 물어보신 부분입니다. 벡터 검색만 쓰지 않고 **세 가지 점수를 가중 합산**합니다.



### 7.1 왜 벡터 검색만으로는 안 되는가

| 검색 방식            | 잘하는 것                                                  | 못하는 것 — 면접에서의 실패                                                                     |
|------------------|--------------------------------------------------------|--------------------------------------------------------------------------------------|
| 벡터만 (× 0.55)     | "동시성 처리 경험" → "락 경험을 줄였다" 같은 표현이 달라도 의미가 같은 문서를 찾음     | UserRepository 를 물었는데 <i>의미상 비슷한 OrderRepository</i> 를 가져옴 → 면접관이 존재하지 않는 코드를 근거로 질문 |
| 어휘만 (× 0.30)     | 단어가 실제로 문서에 있는지 확인                                     | 한국어에서 조사·어미 때문에 신뢰도가 낮음. "구현했습니다"와 "구현"이 다른 토큰                                       |
| 심볼 완전일치 (× 0.80) | 함수·클래스명 직격. <code>lower(symbol) = lower(:query)</code> | 이름을 정확히 모르면 무용지물                                                                     |

#### 가중치 0.80 > 0.55의 의미 — 감리 해석

심볼 완전일치가 벡터 유사도보다 높은 가중치를 받습니다. 코드 면접에서는 **"정확히 그 코드"가 아니면 질문이 성립하지 않기** 때문입니다. 비슷한 코드를 근거로 질문하면 지원자는 "저는 그런 코드를 쓴 적이 없습니다"라고 답할 수밖에 없고, 그 순간 면접 전체의 신뢰가 무너집니다.

반대로 어휘 점수를 0.30으로 낮게 둔 것은 **한국어 형태소 분석기 없이 'simple' 설정을 쓰기** 때문입니다.

`to_tsvector('simple', ...)` 은 한국어를 공백으로만 자릅니다. 이 한계를 알고 가중치를 낮춘 것으로 보이며, 감리에서 **솔직한 설계**로 평가합니다 — 한국어 검색 품질을 과장하지 않았습니다.

### 7.2 LIMIT 200 → 상위 5건

SQL이 200건까지만 후보로 남기고, Python에서 소유권 점수를 더해 재정렬한 뒤 **5건**만 질문 생성에 넘깁니다.

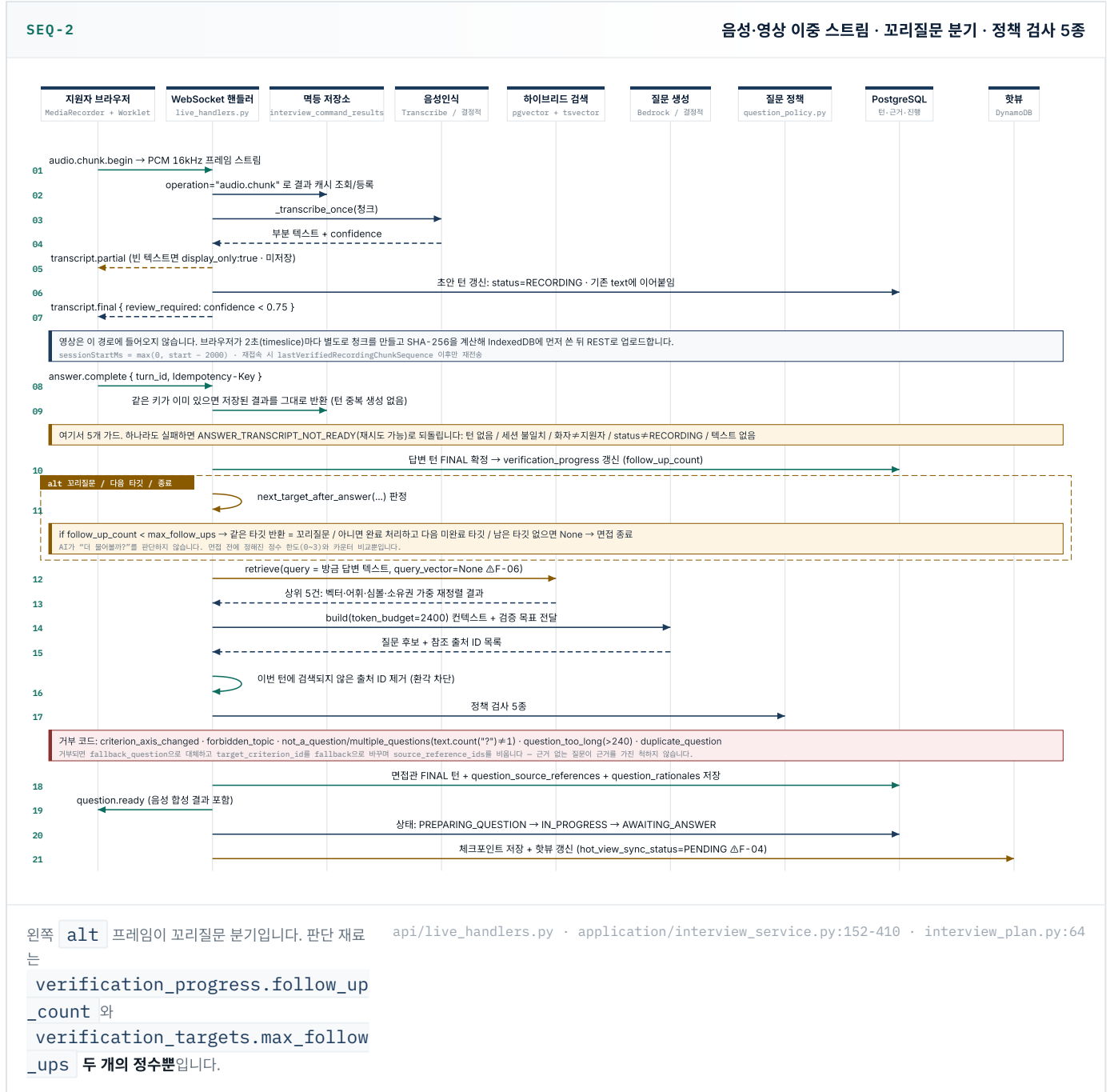
| 단계         | 건수                                  | 이유                                                                              |
|------------|-------------------------------------|---------------------------------------------------------------------------------|
| SQL 1차     | → 200                               | 전체 정렬을 피합니다. 지원자가 대형 저장소를 났을 때 수만 행을 정렬하면 <b>실시간 면접이 멈춥니다</b>                   |
| Python 재정렬 | 200 → 5                             | 소유권 <code>× 0.15</code> 를 더해 "본인이 쓴 코드"를 앞으로. 동점은 <b>안정 정렬</b> 로 입력 순서 유지 → 결정적 |
| 컨텍스트 구성    | <code>token_budget</code><br>= 2400 | 5건이라도 길면 잘라냅니다. 모델 입력 길이가 예측 가능해야 응답 시간이 예측 가능합니다                               |

**F-06 · 이 경로에서 발견한 비효율**

`live_handlers.py:434` 에서 `retrieve(...)` 를 호출할 때 `query_vector=None` 으로 넘깁니다. 그러면 검색 계층이 **매 턴 쿼리 텍스트를 다시 임베딩**합니다. 실시간 경로에서 턴마다 임베딩 왕복 1회가 추가되는 셈입니다. 벡터 길이 검사 (`len(query_vector) != 1024`)가 이미 있으므로 인터페이스는 벡터를 받을 준비가 돼 있습니다 — 호출부에서 미리 계산해 넘기면 해결됩니다. 기능 결함은 아니고 **지연 개선 항목**입니다.

## 08 시퀀스 2 — 실시간 면접 1턴

이 문서에서 가장 중요한 그림입니다. 음성과 영상이 서로 다른 경로로 흐르고, 꼬리질문 분기가 여기서 결정됩니다.



### 8.1 음성과 영상이 다른 경로인 이유

| 스트림 | 경로                                         | 주기                    | 목적                               |
|-----|--------------------------------------------|-----------------------|----------------------------------|
| 음성  | WebSocket<br>AudioWorklet → PCM<br>16kHz   | 프레임 단위<br>(즉시)        | 실시간 인식. 질문 생성을 위해 지연이 없어야 함      |
| 영상  | REST<br>MediaRecorder →<br>IndexedDB → 업로드 | timeslice<br>= 2000ms | 무결성 기록. 실시간성이 필요 없음. 대신 유실되면 안 됨 |

영상을 웹소켓으로 함께 보냈다면 **영상 대역폭이 음성 인식을 지연시킵니다**. 네트워크가 나빠질 때 가장 먼저 희생되어야 할 것은 영상 업로드이고, 절대 희생되면 안 되는 것은 질문·답변 흐름입니다. 경로를 나눠 이 우선순위를 구조로 만들었습니다.

그리고 영상 청크는 **업로드 전에 IndexedDB에 먼저 씁니다**. 업로드가 성공한 뒤에만 로컬 버퍼에서 지웁니다. 브라우저가 죽거나 네트워크가 끊겨도 재접속 후 `lastVerifiedRecordingChunkSequence` 이후 청크만 다시 보내 **중복 없이** 이어집니다.

`sessionStartMs = max(0, start - 2000)` 으로 청크 경계에서 소리가 잘리는 것도 방지합니다.

## 8.2 답변 확정에 걸린 5개 가드

`answer.complete` 가 오면 아래 5개를 모두 통과해야 턴이 `FINAL` 이 됩니다. 하나라도 실패하면 `ANSWER_TRANSCRIPT_NOT_READY` 로 되돌려 **재시도 가능한 상태**로 둡니다.

- ① 그 `turn_id`가 존재하는가
- ② 그 턴이 이 세션의 턴인가 ← 다른 세션의 턴을 확정시키려는 시도 차단
- ③ `speaker`가 **APPLICANT**인가 ← 면접관 턴을 답변으로 확정하려는 시도 차단
- ④ `status`가 **RECORDING**인가 ← 이미 FINAL인 턴 재확정 차단
- ⑤ `text`가 비어 있지 않은가 ← 빈 답변으로는 **FINAL이 되지 않음**

실패 시 → `ANSWER_TRANSCRIPT_NOT_READY` (오류가 아니라 **재시도 가능 상태**)

### ⑤가 있는 이유

음성 인식이 빈 문자열을 돌려줬을 때 **빈 답변을 확정하지 않습니다**. 빈 텍스트가 FINAL로 확정되면 리포트에서 "지원자가 답하지 않았다"가 되어버립니다 — 실제로는 인식이 실패했을 뿐인데도요. 대신 `transcript.partial` 에 `display_only: true` 로 화면에만 표시하고 저장하지 않습니다. **인식 실패와 무응답을 구별하는 설계**입니다.

### 8.3 꼬리질문은 어떻게 결정되는가

물어보신 핵심입니다. 결론부터: **AI가 판단하지 않습니다. 정수 두 개를 비교합니다.**

```
interview_plan.py:64 next_target_after_answer(...)

answered = self.target(answered_target_id)

if follow_up_count < answered.max_follow_ups:
    return answered          ← 같은 타깃을 다시 반환 = 꼬리질문

completed = completed_target_ids | {answered_target_id}
return next((t for t in self.verification_targets
             if t.verification_target_id not in completed), None)
          ← 다음 미완료 타깃, 없으면 None → 면접 종료
```

| 상황                               | 비교         | 결과                                       |
|----------------------------------|------------|------------------------------------------|
| 타깃 A를 처음 물었고<br>max_follow_ups=2 | 0 < 2 → 참  | 타깃 A로 <b>꼬리질문</b> . follow_up_count 가 1로 |
| 같은 타깃 A, 두 번째 꼬리질문 후             | 2 < 2 → 거짓 | A를 완료 처리하고 <b>다음 타깃 B</b> 로 이동           |
| 모든 타깃 완료                         | 남은 타깃 없음   | None 반환 → interview.completed 발행         |

**감리 의견** — 이 단순함이 강점입니다

"AI가 답변의 충분성을 판단해 더 물어볼지 결정한다"는 구조가 아닙니다. 그런 구조라면 **면접 시간을 예측할 수 없고**, 같은 기준으로 지원한 두 사람이 전혀 다른 강도의 면접을 받게 되며, 나중에 "왜 저에게만 더 캐물었나"에 답할 수 없습니다.

지금 구조에서는 max\_follow\_ups 가 면접 전에 정해진 값이므로 **모든 지원자가 같은 상한을 받습니다**. AI의 역할은 "그 한도 안에서 좋은 질문 문장을 만드는 것"으로 좁혀져 있습니다. 다만 이 설계의 대가는 명확합니다 — 답변이 충분해도 한도까지 묻고, 답변이 부실해도 한도를 넘기지 못합니다. **공정성을 위해 적응성을 포기한 트레이드오프**이며, 구조화 면접(structured interview)의 표준 원칙과 일치합니다.

### 8.4 질문 정책 검사 5종 — 거부 코드

모델이 만든 질문은 저장 전에 question\_policy.py 를 통과해야 합니다.

| 거부 코드                                | 검사                    | 막는 사고                                                                        |
|--------------------------------------|-----------------------|------------------------------------------------------------------------------|
| criterion_axis_changed               | 질문이 겨는 기준이 타깃의 기준과 다름 | 동시성을 확인하려던 자리에서 갑자기 리더십을 묻는 <b>평가 축 이탈</b>                                   |
| forbidden_topic                      | 금지 주제 목록 대조           | 나이·출신·가족 등 <b>물어서는 안 되는 질문</b>                                               |
| not_a_question<br>multiple_questions | text.count("?") != 1  | 물음표 0개 = 질문이 아님(설명·평가). 2개 이상 = <b>복합 질문</b> → 어느 답이 어느 질문에 대한 것인지 근거 연결이 깨짐 |
| question_too_long                    | 240자 초과               | 지원자가 질문을 이해하지 못함. 긴 질문은 사실상 복합 질문                                            |
| duplicate_question                   | 이미 한 질문과 중복           | 같은 것을 반복해 물어 <b>면접 시간을 낭비</b>                                                |

거부되면 무엇이 일어나는가 — 여기가 중요합니다

정책이 거부하면 질문을 버리고 `fallback_question` 으로 대체합니다. 이때 **세 가지가 동시에 일어납니다**.

- `target_criterion_id` 를 **fallback으로 변경** — 이 질문이 원래 기준을 겨는 것이 아님을 표시
- `source_reference_ids` 를 **비움** — 근거 없는 질문이 근거를 가진 척하지 않음
- `question_rationales.policy_result` 에 **거부 사실이 기록됨**

감리에서 높게 평가하는 지점입니다. 대체 질문에 원래 질문의 근거를 그대로 남겨두면, 나중에 리포트를 볼 때 **"이 질문은 이 코드를 근거로 했다"**는 거짓 기록이 됩니다. 출처를 비우는 처리가 그것을 막습니다. 그리고 거부 사실 자체가 남으므로 **모델이 얼마나 자주 부적절한 질문을 만드는지**를 사후에 측정할 수 있습니다.

## 8.5 환각 출처 차단

### RETRIEVED\_BY\_ID 필터

모델이 반환한 출처 ID 중 **그 턴에서 실제로 검색된 ID 집합에 없는 것은 제거**합니다. 모델이 그럴듯한 파일 경로나 문서 ID를 만들어내도 `question_source_references` 에 저장되지 않습니다. RAG 시스템에서 가장 흔한 사고 — *"출처를 붙였지만 그 출처는 존재하지 않음"* — 를 코드로 막은 것입니다. 이 시스템 전체에서 **단일 최고 설계**로 평가합니다.

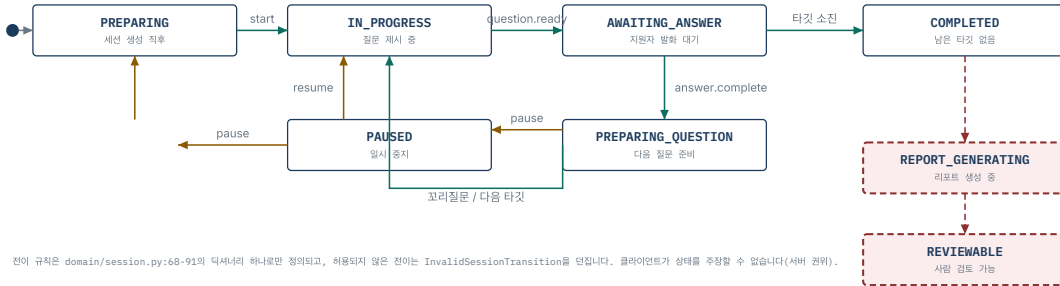


## 09 세션 상태 기계와 복구

세션 상태는 **서버만** 바꿉니다. 클라이언트가 "나는 지금 답변 중이다"를 주장할 수 없습니다.

FIG-9

8개 상태와 허용된 전이 — 점선은 호출되지 않는 전이



전이 규칙은 domain/session.py:68-91의 디세너리 하나로만 정의되고, 허용되지 않은 전이는 InvalidSessionTransition를 던집니다. 클라이언트가 상태를 주장할 수 없습니다(서버 권위).

△ F-03 · 점선 두 전이는 도메인에 정의만 되어 있고 이를 호출하는 코드가 없습니다. 세션은 COMPLETED에서 영구히 멈춥니다.  
리포트 위키는 reports 행을 만들지만 세션 상태를 바꾸지 않습니다 → "리포트가 준비됐는가"를 세션 상태로 반영할 수 없고, 콘솔은 reports를 직접 조회해야 합니다.

전이 규칙이 **딕셔너리 하나**로만 정의되어 있어 감리가 쉽습니다. 허용 목록에 없는 전이는 `InvalidSessionTransition` 을 던집니다.

interview\_engine/domain/session.py:68-91

| 상태                 | 허용된 다음 상태                               | 감리 확인                                                |
|--------------------|-----------------------------------------|------------------------------------------------------|
| PREPARING          | IN_PROGRESS                             | 실행됨                                                  |
| IN_PROGRESS        | AWAITING_ANSWER · PAUSED · COMPLETED    | 실행됨                                                  |
| AWAITING_ANSWER    | PREPARING_QUESTION · PAUSED · COMPLETED | 실행됨                                                  |
| PREPARING_QUESTION | IN_PROGRESS · PAUSED                    | 실행됨                                                  |
| PAUSED             | IN_PROGRESS                             | 실행됨. <b>PAUSED에서 COMPLETED로 갈 수 없음</b> — 재개 없이 종료 불가 |
| COMPLETED          | REPORT_GENERATING                       | 호출 0건 · F-03                                         |
| REPORT_GENERATING  | REVIEWABLE                              | 호출 0건 · F-03                                         |
| REVIEWABLE         | — (종단)                                  | 도달 불가                                                |

**PAUSED → COMPLETED가 없는 것은 의도로 봅니다**

일시 중지 상태에서 바로 종료할 수 없습니다. 반드시 `IN_PROGRESS` 로 돌아와야 합니다. 네트워크가 끊긴 채로 세션이 "완료"되어 **지원자가 답변 기회를 잃는 것**을 막는 제약입니다. 상태 기계로 이런 정책을 표현한 것은 좋은 선택입니다 — `if` 문으로 흩어져 있으면 한 곳을 놓칩니다.

COMPLETED → REPORT\_GENERATING → REVIEWABLE 전이가 도메인에 정의만 되어 있고 이를 호출하는 코드가 없습니다. 리포트 워커는 reports 행을 만들지만 세션 상태를 바꾸지 않습니다. 결과적으로 "리포트가 준비됐는가"를 세션 상태로 알 수 없고, 기업 콘솔은 reports 테이블을 직접 조회해야 합니다. 권고: 리포트 워커 시작 시 REPORT\_GENERATING, 저장 완료 시 REVIEWABLE 로 전이시키면 이미 있는 전이표로 해결됩니다.

## 9.2 DynamoDB는 무엇을 저장하는가

물어보신 "다이나모디비 저장"입니다. DynamoDB는 **주 저장소가 아닙니다**. 세션 하나당 **단 1건의** 항목만 두는 **핫뷰(hot view)**입니다.

키 PK = "SESSION#<interview\_session\_id>" SK = "META"

→ 세션 하나에 항목 **하나**. 목록 조회(Scan/Query) 없음.

항목 company\_id · interview\_session\_id · session\_sequence  
checkpoint\_id · last\_media\_chunk\_sequence · schema\_version  
last\_final\_turn\_id? · pending\_turn\_id? · last\_reconciled\_event\_id?  
**ttl** ← 숫자. 체크포인트 created\_at + 7일

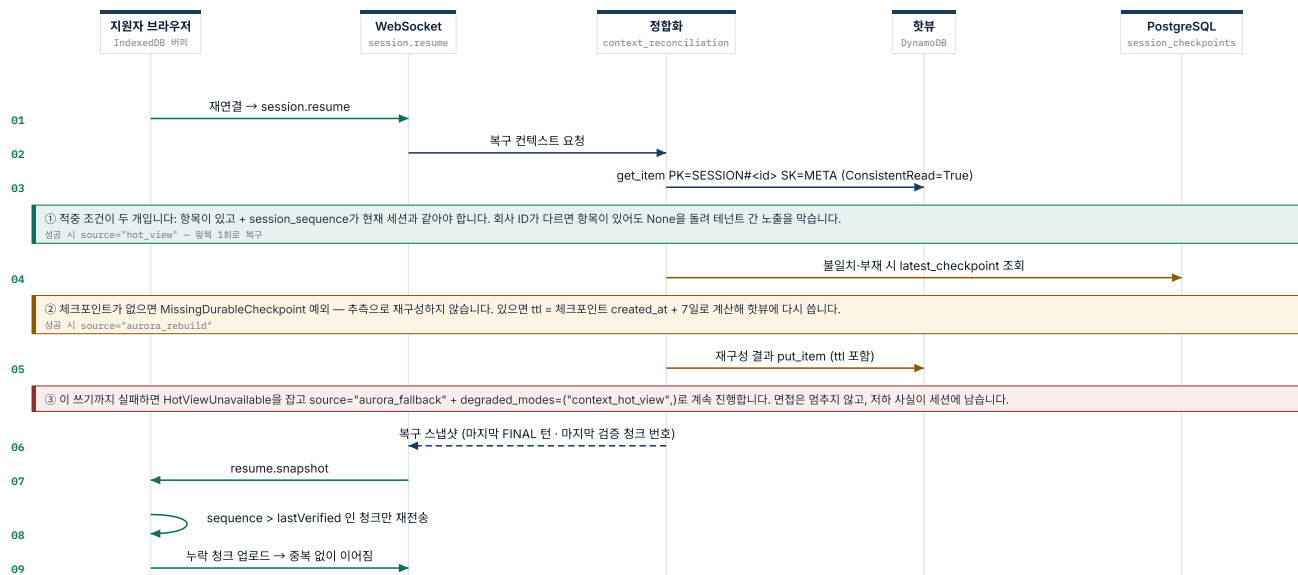
읽기 get\_item(**ConsistentRead=True**)

→ 최종 일관성 읽기를 쓰지 않습니다. 복구 시점에 **오래된 값**을 읽으면  
이미 확정된 턴을 다시 요구하게 됩니다.

| DYNAMODB에 있는 것    | POSTGRESQL에 있는 것        | 왜 나눴는가                                                             |
|-------------------|-------------------------|--------------------------------------------------------------------|
| 마지막 복구점 <b>1건</b> | 모든 턴 · 모든 체크포인트 · 모든 청크 | 복구는 <b>가장 최근 1건만</b> 필요합니다. 재접속마다 관계형 DB에 조인 질의를 던지면 실시간 경로가 느려집니다 |
| TTL로 <b>자동 소멸</b> | 보존 정책에 따라 관리            | 핫뷰는 캐시이므로 7일 후 사라져도 무해합니다. 원본은 PostgreSQL에 있습니다                    |
| 없어도 <b>동작함</b>    | 없으면 복구 불가               | DynamoDB 장애가 면접을 중단시키지 않도록 설계 — 아래 3단 폴백                           |

FIG-10

재접속 복구 시퀀스 — 핫뷰 3단 폴백



세 단계 모두 구현되어 있고, 마지막 단 application/context\_reconciliation.py:45 · adapters/recent\_context.py · features/interview/media.ts 계에서도 면접이 계속됩니다. 대신

`degraded_modes`에 저하 사실이 남습니다.

| 단 | SOURCE 값                        | 조건                                                   | 결과                                                                   |
|---|---------------------------------|------------------------------------------------------|----------------------------------------------------------------------|
| ① | hot_view                        | 항목 있음 AND <code>session_sequence</code> 일치           | 왕복 1회로 복구. 가장 빠름                                                     |
| ② | aurora_rebuild                  | 핫뷰 미스 → <code>latest_checkpoint</code> 조회 성공         | PostgreSQL에서 재구성하고 <b>핫뷰에 다시 씀</b> (ttl 재계산)                         |
| ③ | aurora_fallback                 | 재구성 후 핫뷰 쓰기까지 실패 ( <code>HotViewUnavailable</code> ) | 면접 <b>계속 진행</b> + <code>degraded_modes= ("context_hot_view",)</code> |
| × | <b>MissingDurableCheckpoint</b> | 체크포인트 자체가 없음                                         | <b>예외.</b> 추측으로 재구성하지 않음                                             |

감리 의견 — 세 가지가 모두 맞습니다

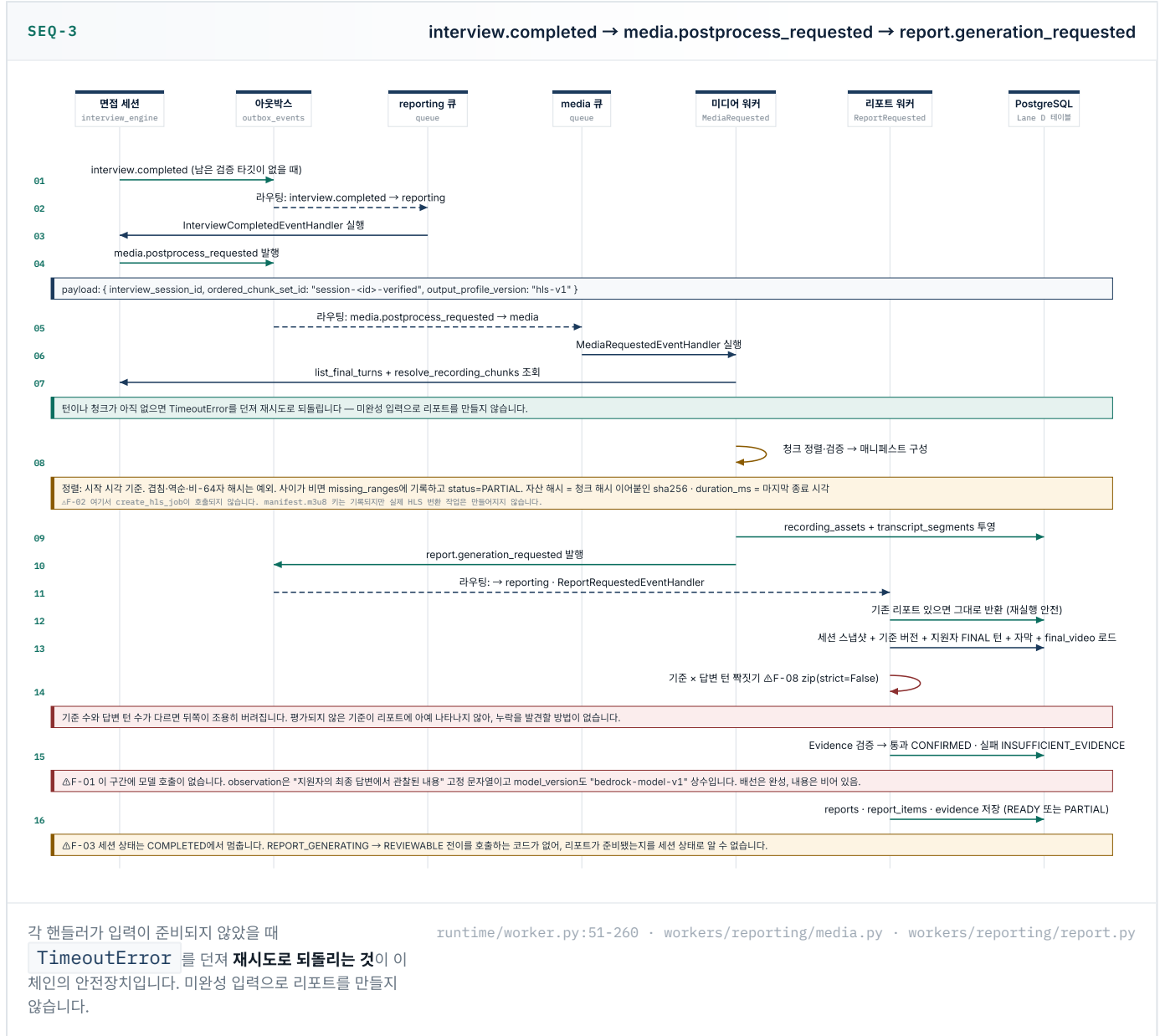
- 모든 boto 예외를 `HotViewUnavailable` 로 감쌉니다. DynamoDB의 구체적 오류 유형이 면접 로직으로 새지 않습니다 — 어댑터 경계가 지켜집니다
- 회사 ID가 다르면 항목이 있어도 `None` 을 반환합니다. 핫뷰에도 테넌시가 적용됩니다. 캐시가 우회 경로가 되지 않습니다
- 저하를 숨기지 않습니다. `degraded_modes` 가 세션에 남으므로 "이 면접은 핫뷰 없이 진행됐다"가 기록됩니다. 조용히 느껴지는 것보다 훨씬 낫습니다

F-04 · 다만 동기 상태 값은 신뢰할 수 없습니다

`session_checkpoints.hot_view_sync_status` 는 `PENDING / SYNCED / DEGRADED` 세 값을 가질 수 있는데, 코드에서 쓰는 값은 항상 `PENDING` 입니다. `SYNCED` 로 바뀌는 경로는 테스트 코드에만 있습니다. 핫뷰 쓰기 성공 후 `SYNCED` , `HotViewUnavailable` 시 `DEGRADED` 로 갱신하면 **운영 지표로 쓸 수 있는 값**이 됩니다. 지금은 이 컬럼으로 아무것도 판단할 수 없습니다.

## 10 시퀀스 3 — 면접 후 워커 체인

이벤트 3개가 연쇄합니다. 배선은 완성되어 있고, 마지막 단계의 내용이 비어 있습니다.



### 10.1 재시도 안전성 — 3개 핸들러 전부 확인

| 핸들러                            | 중복 실행 시                                                                               | 미준비 입력 시                                                                                       |
|--------------------------------|---------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------|
| InterviewCompletedEventHandler | idempotency_key = "media-postprocess-<session_id>" 가 고정값이므로 아웃박스 UNIQUE 제약이 중복 발행을 차단 | —                                                                                              |
| MediaRequestedEventHandler     | 같은 object_key 를 덮어씀. 입력이 같으면 출력이 같음(결정적)                                              | TimeoutError("completed session projection is not ready") → 재시도                                |
| ReportRequestedEventHandler    | 기존 리포트가 있으면 그대로 반환하고 아무것도 만들지 않음                                                      | TimeoutError("report inputs are not ready") / ("report transcript inputs are not ready") → 재시도 |

입력이 아직 준비되지 않은 것은 **오류가 아니라 타이밍**입니다. 이때 일반 `Exception` 을 던지면 워커가 이를 실패로 기록하고 DLQ로 보내거나 알람을 냅니다. `TimeoutError` 로 구분해 던지므로 **"나중에 다시 해봐라"**가 정확히 전달됩니다. 미디어 워커에서는 턴이나 청크가 하나도 없을 때 이 예외가 나가는데, 실제로 `interview.completed` 처리가 청크 업로드보다 빠를 수 있으므로 반드시 필요한 처리입니다.

## 10.2 미디어 워커가 하는 검증

정렬      `session_start_ms` 기준 오름차순  
거부 조건    청크 구간 겹침 · 역순 · `content_hash` 길이  $\neq 64$  → 예외  
빈 구간    청크 사이에 공백 → `missing_ranges` JSON에 기록 + `status = PARTIAL`  
자산 해시    각 청크 해시를 순서대로 이어붙인 문자열의 `sha256`  
`duration`    마지막 청크의 `session_end_ms`  
출력 키      `companies/<company_id>/sessions/<id>/recording/final/v1/manifest.m3u8`

### 빈 구간을 숨기지 않는 것이 중요합니다

녹화 중간이 비었을 때 그것을 조용히 이어붙이면 **"연속된 녹화"**처럼 보이는 편집된 영상이 됩니다. `missing_ranges` 에 기록하고 `status=PARTIAL` 로 강등하므로, 리포트를 보는 사람이 **"이 영상에는 빠진 구간이 있다"**를 알 수 있습니다. 근거 자료로서 영상이 갖춰야 할 최소 조건입니다.

### F-02 · 다만 HLS 변환이 실행되지 않습니다

출력 키에 `manifest.m3u8` 가 기록되지만, 실제로 HLS를 만드는 `create_hls_job` 은 **단위 테스트에서만 호출**됩니다. 미디어 워커 본문에 호출 지점이 없습니다. 결과적으로 `recording_assets` 에는 존재하지 않는 매니페스트를 가리키는 키가 남습니다. Evidence 의 `video_start_ms` / `video_end_ms` 컬럼 자체는 채워지지만, **그 값이 가리키는 시각이 맞는지**는 별개의 문제입니다 — 아래 F-09를 보십시오. 콘솔에서 해당 구간을 재생하려 하면 자산이 없어 실패합니다.

### 10.3 F-09 · 답변 턴과 영상 청크를 위치로 짝짓습니다

미디어 워커가 각 답변 턴의 영상 구간을 정하는 코드입니다( `runtime/worker.py:126-132` ).

```
turns = list_final_turns(...) # 답변 턴 목록
chunks = resolve_recording_chunks(...) # verified 청크 목록 (2초 단위)

FinalTurnRange(
    turn_id = turn.turn_id,
    session_start_ms = chunks[min(index, len(chunks) - 1)].session_start_ms,
    session_end_ms = chunks[min(index, len(chunks) - 1)].session_end_ms,
    confidence = 0.9,
)
for index, turn in enumerate(turns)
```

**N번째 답변 턴에 N번째 2초 청크의 시각이 들어갑니다**

프론트엔드의 `timesliceMs = 2000` ( `features/interview/media.ts:65` )이므로 청크 하나는 2초입니다. 40초짜리 답변은 청크 약 20개에 걸칩니다. 그런데 위 코드는 턴 인덱스를 청크 인덱스로 그대로 씁니다.

| 답변 턴 | 실제 발언 구간           | 코드가 넣는 구간                                | 결과        |
|------|--------------------|------------------------------------------|-----------|
| 1번째  | 0 ~ 40,000ms       | <code>chunks[0] = 0 ~ 2,000ms</code>     | 시작만 맞음    |
| 2번째  | 45,000 ~ 90,000ms  | <code>chunks[1] = 2,000 ~ 4,000ms</code> | 완전히 다른 시각 |
| 3번째  | 95,000 ~ 130,000ms | <code>chunks[2] = 4,000 ~ 6,000ms</code> | 완전히 다른 시각 |

두 번째 턴부터 어긋나고, 면접이 길어질수록 오차가 커집니다. 이 값이

`transcript_segments.session_start_ms / session_end_ms` 로 투영되고, 그 세그먼트가 다시 `evidence.video_start_ms / video_end_ms` 의 기준이 됩니다.

**왜 EVIDENCE 검증이 이것을 잡지 못하는가**

`validate_timeline` 은 영상 구간이 자기 자막 세그먼트 안에 들어있는지를 봅니다. 그런데 그 자막 세그먼트의 시각도 *같은 잘못된* 값에서 왔습니다. *잘못된 기준에 대해 일관되므로 검증을 통과합니다.* 결과적으로 `CONFIRMED` 가 붙은 Evidence가 *엉뚱한 시각*을 가리킬 수 있습니다.

이것이 F-02(재생 불가)보다 무거운 이유는, 재생이 안 되는 것은 보이는 고장이지만 이것은 재생이 붙는 순간 조용히 틀린 장면을 보여주는 것이기 때문입니다. 검토자는 "AI가 근거로 든 발언"을 확인하려고 그 시각으로 이동했는데 다른 발언을 듣게 됩니다.

고칠 데이터는 이미 저장되어 있습니다

추가 수집이 필요한 문제가 아닙니다. 클라이언트는 답변을 확정할 때마다 `last_recording_chunk_sequence` 를 함께 보내고 ( `api/live_handlers.py:283` ), 서버는 그것을 `session_checkpoints` 에 ( `last_final_turn_id`, `last_media_chunk_sequence` ) **쌍으로 기록합니다** ( `interview_service.py:265-272` · `398-405` · `463-470` ). 즉 어느 턴이 어느 청크 번호까지 진행했는지가 이미 durable하게 남아 있습니다.

**권고:** 미디어 워커가 `chunks[index]` 대신 이 체크포인트 쌍으로 **턴 경계를 복원**하도록 바꿉니다 (현재 `runtime/worker.py` 와 `integration/interview_reporting.py` 에 `checkpoint` 참조가 **0건**입니다). 더 견고하게 하려면 `interview_turns` 에 `session_start_ms` / `session_end_ms` 를 직접 두는 편이 낫습니다 — 현재 이 테이블에는 **시각 컬럼이 하나도 없어** 시간 기준 조인 자체가 불가능하고, 그래서 위치 짝짓기가 나온 것으로 보입니다.



## 10.4 리포트 생성기가 실제로 하는 일

workers/reporting/report.py:38 ReportGenerator.generate 를 한 줄씩 확인한 결과입니다.

### 기준마다 반복

#### ① Evidence 후보 구성

```
Evidence(answer_turn_id, transcript_segment_id, video_start_ms, video_end_ms,
          rationale = "최종 답변의 자막과 유효 영상 구간에 직접 연결됨" ← 고정 문자열
          sufficiency = Sufficiency.DIRECT,
          generation_version = "report-v1")
```

#### ② evidence\_service.validate(...) ← 실제 검증. 여기는 동작합니다

```
└ 성공 → AssessmentState.CONFIRMED
  |   uncertainty = "AI 원본이며 사람 검토 필요"
└ (EvidenceRangeError, ValueError) → AssessmentState.INSUFFICIENT_EVIDENCE
   uncertainty = "유효한 영상·자막 Evidence가 부족함"
```

### 리포트 조립

```
Report(kind = ReportKind.AI_ORIGINAL, ← 사람 수정과 구별
        model_version = "bedrock-model-v1", ← 상수
        prompt_version = "report-prompt-v1", ← 상수
        config_version = "report-config-v1", ← 상수
        status = READY if 모든 항목에 evidence else PARTIAL,
        summary = "최종 답변 Evidence에 기반한 AI 원본 리포트") ← 고정 문자열
```

### F-01 · 모델 호출이 없습니다

이 함수 안에 모델을 부르는 코드가 한 줄도 없습니다. report\_items.observation 은 ReportRequestedEventHandler 에서 CriterionInput(observation="지원자의 최종 답변에서 관찰된 내용", ...) 으로 만들어지는 고정 문자열이고, rationale · summary 도 상수입니다.

다만 동작하는 것과 동작하지 않는 것을 정확히 구분해야 합니다.

| 구분                                           | 상태                                                                        |
|----------------------------------------------|---------------------------------------------------------------------------|
| Evidence 검증<br>( evidence_service.validate ) | 동작합니다. 답변 턴·자막·영상 구간이 서로 맞는지 실제로 검사하고, 실패하면 INSUFFICIENT_EVIDENCE 로 떨어집니다 |
| Evidence 연결 — 턴 ID · 자막 ID                   | 동작합니다. 실제 데이터로 채워집니다                                                      |
| Evidence 연결 — 영상 시각 구간                       | 채워지지만 값이 틀립니다(F-09). 검증은 이 값을 기준으로 삼으므로 통과합니다                             |
| 리포트 상태 판정 (READY / PARTIAL)                  | 동작합니다. 모든 항목에 Evidence가 있는지로 결정                                           |
| 평가 내용 생성 ( observation · rationale )         | 고정 문자열. 모델 미연동                                                            |

즉 "근거를 검증하는 골격"은 완성이고 "내용을 쓰는 부분"이 비어 있습니다. 로컬 런타임이 LocalDeterministicModel 로 완전 결정적으로 동작하도록 만들어져 있다는 점을 보면, 테스트를 먼저 안정화하고 모델을 나중에 붙이는 순서로 보입니다. 골격이 먼저 완성된 것은 오히려 좋은 순서입니다 — 모델을 붙일 때 검증 로직을 새로 만들 필요가 없습니다.

10.5 F-08 · 조용히 누락되는 짝짓기

```

criterion_inputs = [
    CriterionInput(criterion_id=c.criterion_id, answer_turn_id=t.turn_id, ...)
    for c, t in zip(criterion.criteria, turns, strict=False)
]

```

| 기준 수 | 답변 턴 수 | 결과                                |
|------|--------|-----------------------------------|
| 5    | 5      | 정상. 5개 항목                         |
| 5    | 3      | 3개 항목만 생성. 기준 2개가 리포트에 나타나지 않음    |
| 3    | 5      | 3개 항목만 생성. 답변 2개가 어느 기준에도 연결되지 않음 |

왜 이것을 F-09 다음으로 먼저 닫으라고 권고하는가

나머지 6건은 **증상이 보입니다**. F-01은 리포트 내용이 고정 문자열이라 바로 눈에 띄고, F-02는 영상 재생이 실패하며, F-03은 상태가 안 바뀌는 것이 보입니다. **증상이 없는 것은 F-09와 F-08 둘뿐입니다** — 그래서 이 둘이 앞순위입니다.

F-08은 **아무 증상이 없습니다**. 예외도 없고 로그도 없습니다. 기준 5개 중 3개만 평가된 리포트가 *정상적인 3개 항목 리포트*처럼 보입니다. 평가되지 않은 기준은 **리포트에 아예 나타나지 않으므로** 검토자가 "이 기준이 빠졌다"를 알아챌 방법이 없습니다. `status` 도 `READY` 로 남습니다 — 생성된 3개 항목에는 모두 Evidence가 있으니까요.

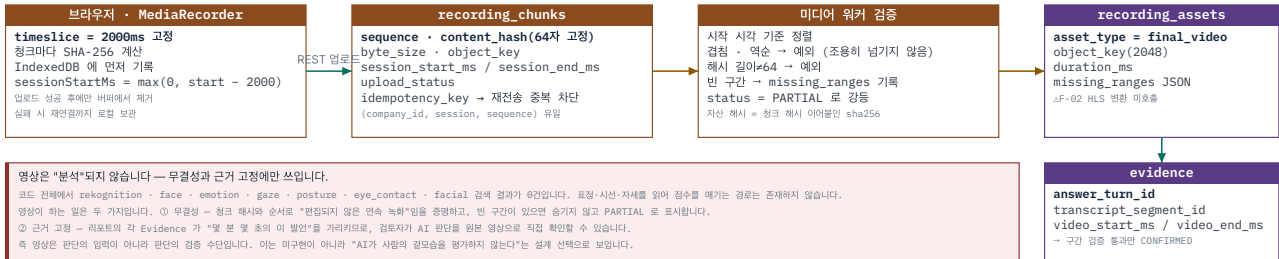
**권고:** `strict=True` 로 바꿔 예외를 내거나, 수가 다를 때 남는 기준을 `INSUFFICIENT_EVIDENCE` 항목으로 명시적으로 만들어야 합니다. 후자가 더 낫습니다 — 답변이 부족했다는 사실 자체가 리포트에 남아야 하는 정보입니다.

## 11 영상은 어떻게 들어가는가

물어보신 부분입니다. 결론: 영상 "분석"은 존재하지 않습니다. 영상은 무결성 증명과 근거 고정에만 쓰입니다.

FIG-11

무결성 · 근거 고정 전용 경로



영상은 판단의 **입력**이 아니라 판단의 **검증 수단** `features/interview/media.ts · workers/reporting/media.py · reporting/application/evidence.py`입니다. 이 부분이 이 시스템의 성격을 정의합니다.

### 11.1 없다는 것을 어떻게 확인했는가

추측이 아니라 전수 검색으로 확인했습니다. 저장소 전체에서 다음 7개 키워드의 검색 결과가 **0건**입니다.

| 검색어                | 결과 | 있었다면 무엇을 의미했을까       |
|--------------------|----|----------------------|
| rekognition        | 0  | AWS 영상·이미지 분석 서비스 연동 |
| face · facial      | 0  | 얼굴 검출·인식             |
| emotion            | 0  | 표정으로 감정 추정           |
| gaze · eye_contact | 0  | 시선 추적 · 눈맞춤 측정       |
| posture            | 0  | 자세 분석                |

### 11.2 그러면 영상은 무엇을 하는가

#### ① 무결성 — 편집되지 않았음의 증명

청크마다 SHA-256과 순번과 시각 구간이 남습니다. 미디어 워커가 이를 정렬하며 겹침·역순을 예외로 처리하고, 빈 구간은 `missing_ranges` 에 기록합니다. 최종 자산 해시는 **청크 해시들을 순서대로 이어붙인 sha256**이므로 한 청크만 바뀌어도 전체 해시가 달라집니다.

#### ② 근거 고정 — 구조는 갖췄지만 시각이 어긋납니다

리포트의 각 `evidence` 행이 `answer_turn_id + transcript_segment_id + video_start_ms / video_end_ms` 를 동시에 가리킵니다. 구조적으로는 **몇 분 몇 초의 그 발언**으로 되돌아갈 수 있게 설계됐고, 구간 검증에 실패하면 `CONFIRMED` 가 되지 않습니다. 다만 그 시각을 정하는 지점이 턴을 청크와 *위치*로 짝짓고 있어 (F-09) **현재 값은 두 번째 턴부터 실제 발언 시각과 다릅니다**. 턴 ID와 자막 연결은 맞으므로 어느 *발언인지는* 특정되지만, *영상에서 몇 초인지는* 아직 신뢰할 수 없습니다.

감리 의견 — 이것은 미구현이 아니라 설계 선택으로 봅니다

근거가 셋입니다.

- **Evidence 구조가 그것을 허용하지 않습니다.** `evidence` 는 반드시 `answer_turn_id` (지원자의 최종 답변 턴)를 가리켜야 합니다. "표정이 불안해 보였다"는 관찰은 어떤 답변 턴에도 연결되지 않으므로 이 구조에 들어갈 자리가 없습니다.
- **검증 타겟 5종이 모두 말한 내용에 관한 것입니다.** 모순·미언급·소유권·누락 차원·주장 — 겹모습에 관한 유형이 하나도 없습니다.
- **영상 처리 파이프라인이 프레임을 열지 않습니다.** 미디어 워커는 청크를 *바이트와 해시와 시각*으로만 다룹니다. 디코딩 자체가 없습니다.

표정·시선·자세로 사람을 평가하는 기능은 여러 나라에서 법적·윤리적 문제가 제기되는 영역입니다. 그것을 **구현했다가 끄는 것과 애초에 들어갈 자리를 만들지 않는** 것은 다릅니다. 후자입니다.

확인해 두실 점 — F-09 · F-02 · F-07이 같은 목적을 막고 있습니다

위 ②(근거 고정)가 실제로 성립하려면 세 가지가 다 필요한데, 셋 다 아직입니다.

| ID   | 막고 있는 것                         | 결과                             |
|------|---------------------------------|--------------------------------|
| F-09 | 턴↔청크를 위치로 찍지음                   | 시각 값 자체가 틀림. 재생이 붙으면 다른 발언이 나옴 |
| F-02 | <code>create_hls_job</code> 미호출 | 재생 자산이 없어 이동 자체가 불가            |
| F-07 | <code>confidence=0.9</code> 고정  | 음성 인식이 불확실했던 구간을 검토자가 구별 못 함   |

**순서가 중요합니다.** F-02만 먼저 닫으면 *재생은 되는데 틀린 장면을 보여주는* 상태가 됩니다 — 지금보다 나쁩니다. 시각을 확정할 수 없는 동안에는 오히려 재생 버튼이 없는 편이 안전하므로, **F-09를 먼저 닫고 그 다음 F-02**를 연결하는 순서를 권고합니다.

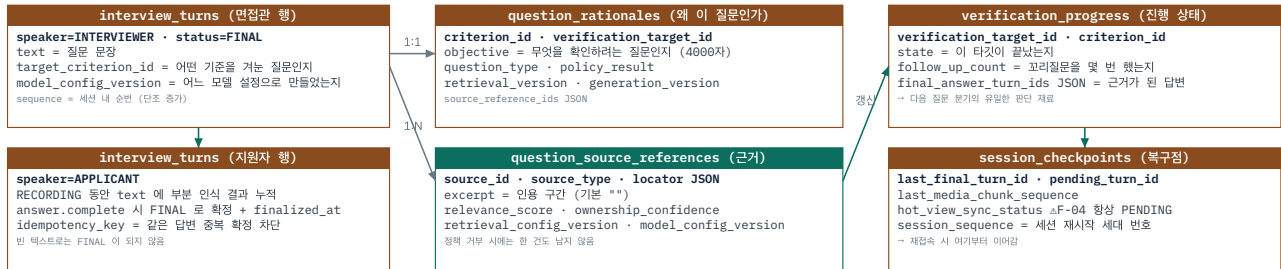
## 12 질문·꼬리질문·답변의 저장 구조

물어보신 "내부적으로 어떻게 저장·분석되는가"입니다. 질문 문장과 그 질문의 근거가 다른 테이블에 있습니다.

FIG-12

한 번의 질문 · 한 번의 답변이 남기는 행

한 번의 질문 · 한 번의 답변이 남기는 행



설계의 핵심: 질문 문장과 "그 질문의 근거"가 서로 다른 테이블에 있습니다.

(면접이 끝난 뒤 "이 질문은 무엇을 보려고 했고, 어느 제출물의 어느 위치를 근거로 삼았는가")를 추측 없이 재구성할 수 있습니다. 리포트의 Evidence 도 같은 원리로 답변 ID + 지막 세그먼트 ID + 영상 구간(ms)을 함께 저장합니다.

문물 번호: 제출물 원문은 `submission_chunks.text_object_key`(객체 키)로만 참조되고, `retrieval_documents`는 `protected_text`(원문)와 `search_text`(검색문)를 분리해 둡니다.

지원자 답변 텍스트와 원문은 로그·설정에 남기지 않는다는 규칙이 README에 명시되어 있고, 오류 응답 경우에는 `request_id` 만 담깁니다.

`interview_turns` 는 대화 내용만 담습니다. "왜 물었나"는 `question_rationales`, "무엇을 근거로"는 `question_source_references`, "얼마나 진행됐나"는 `verification_progress` 가 담당합니다.

`interview_engine/repositories/postgres.py` · `reporting/repositories/postgres.py`

### 12.1 꼬리질문은 어떻게 구별되는가

중요한 점: 꼬리질문을 표시하는 컬럼이 없습니다. 대신 두 값으로 판별됩니다.

| 판별 재료                                                                    | 어떻게 알 수 있는가                                                        |
|--------------------------------------------------------------------------|--------------------------------------------------------------------|
| <code>question_rationales</code><br><code>.verification_target_id</code> | 연속된 두 질문이 같은 <code>verification_target_id</code> 를 가지면 뒤쪽이 꼬리질문입니다 |
| <code>verification_progress</code><br><code>.follow_up_count</code>      | 그 타깃에 대해 몇 번째 꼬리질문인지. 면접 종료 시점의 값이 곧 "몇 번 캐물었는가"                   |

감리 의견 — 별도 플래그를 두지 않은 것이 맞습니다

`is_follow_up` 같은 컬럼을 두면 진실이 두 곳에 생깁니다 — 플래그와 `verification_target_id` . 둘이 어긋나면 어느 쪽이 맞는지 알 수 없습니다. 지금 구조에서는 `verification_target_id` 가 유일한 진실이고, 꼬리질문 여부는 그것에서 파생됩니다. 정규화가 잘 지켜졌습니다.

### 12.2 답변은 어떻게 "분석"되는가

답변 텍스트에 대해 감정 분석이나 점수화를 하지 않습니다. 답변이 하는 일은 두 가지입니다.

| 용도           | 사용 위치                                                                      | 구체적으로                                                                           |
|--------------|----------------------------------------------------------------------------|---------------------------------------------------------------------------------|
| 다음 검색의 질의문   | <code>live_handlers.py:434</code><br>→ <code>retrieve(query=답변 텍스트)</code> | 방금 한 답변을 그대로 검색어로 써서 관련 제출물 근거를 찾습니다. 지원자가 언급한 내용에 이어지는 질문이 나오는 이유입니다           |
| Evidence의 앵커 | <code>verification_progress</code><br><code>.final_answer_turn_ids</code>  | 어느 답변이 어느 타깃의 근거가 됐는지 기록. 리포트의 <code>evidence.answer_turn_id</code> 가 이것을 가리킵니다 |

답변을 점수화하지 않는 것의 의미

"이 답변은 7점"이라는 중간 숫자가 어디에도 저장되지 않습니다. 리포트의 판정은 `CONFIRMED` / `INSUFFICIENT_EVIDENCE` 같은 상태이고, 그 상태마다 근거로 되돌아갈 링크가 붙습니다. 숫자 점수는 근거 없이도 만들어지고 비교되기 쉬운데, 상태 + 링크 구조는 근거 없이는 만들어질 수 없습니다. AI가 최종 결정에 개입할 경로를 구조적으로 없앤 것과 같은 방향의 설계입니다.

12.3 보호 대상 텍스트가 어디에 있는가

| 텍스트 종류  | 저장 위치                                                                                        | 보호 방식                                                  |
|---------|----------------------------------------------------------------------------------------------|--------------------------------------------------------|
| 제출물 원문  | 객체 저장소<br>( <code>submission_chunks.text_object_key</code> )                                 | DB에는 <b>키만</b> . 본문은 관계형 DB에 들어오지 않음                   |
| 검색용 텍스트 | <code>retrieval_documents</code><br><code>.protected_text</code> / <code>.search_text</code> | 원문과 검색용을 <b>분리</b> . 검색은 <code>search_text</code> 만 사용 |
| 답변 텍스트  | <code>interview_turns.text</code><br><code>transcript_segments.text</code>                   | 근거 제시에 필요하므로 저장. <b>로그·설정·오류 응답에는 금지</b>               |
| 초대 토큰   | <code>invitations.token_hash(64)</code><br><code>applicant_access_tokens</code>              | 원문 미저장. <b>해시만</b>                                     |
| 오류 응답   | <code>error envelope</code>                                                                  | <code>request_id</code> 만 포함. 보호 텍스트 없음                |

## 13 발견 사항 9건과 우선순위

9건 모두 재현 조건과 영향, 권고 조치를 함께 적었습니다. 이 중 1건(F-09)은 **데이터가 틀리는** 항목입니다.

### 등급 기준

- **차단** — 저장된 값 자체가 틀리며, 검증을 통과해 **맞는 것처럼 보이는 것** (F-09)
- **가장 먼저 달을 것** — 증상이 보이지 않아 나중에 원인 추적이 어려운 것 (F-08)
- **운영 전 필수** — 실제 채용에 쓰기 전에 반드시 달아야 하는 것 (F-01·F-02·F-03·F-04)
- **개선** — 지금 상태로도 동작하며, 성능·관측성 개선 항목 (F-05·F-06·F-07)

### F-09 답변 턴의 영상 구간을 턴 인덱스 = 청크 인덱스로 정한다

|               |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |
|---------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 등급            | 차단 — 이 문서에서 유일하게 <b>저장된 값이 틀린</b> 항목입니다                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                |
| 위치            | <code>runtime/worker.py:126-132</code> <code>MediaRequestedEventHandler.__call__</code>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                |
| 재현 조건         | <code>chunks[min(index, len(chunks) - 1)]</code> — <code>enumerate(turns)</code> 의 턴 순번을 그대로 청크 배열 첨자로 씁니다. 프런트엔드 <code>timesliceMs = 2000 ( features/interview/media.ts:65 )</code> 이라 청크 하나는 2초입니다. 답변이 2초를 넘는 순간부터 두 인덱스가 어긋나며, <b>두 번째 답변 턴부터 실제 발언과 무관한 시각이 들어갑니다</b> .                                                                                                                                                                                                                                                                                                                                                                           |
| 영향            | 이 값이 <code>transcript_segments.session_start_ms / session_end_ms</code> 로 투영되고, 다시 <code>evidence.video_start_ms / video_end_ms</code> 의 기준이 됩니다. <b><code>validate_timeline</code> 은 이것을 잡지 못합니다</b> — 영상 구간이 <b>자기 자막 세그먼트 안에 있는지</b> 를 보는데 그 세그먼트의 시각도 같은 잘못된 값에서 왔기 때문입니다. 잘못된 기준에 대해 일관되므로 <code>CONFIRMED</code> 가 붙습니다.                                                                                                                                                                                                                                                                                                                         |
| 왜 F-02보다 무거운가 | 재생이 안 되는 것은 <b>보이는 고장</b> 입니다. 이것은 <b>재생이 붙는 순간 조용히 틀린 장면을 보여주는</b> 고장입니다. 검토자는 "AI가 근거로 든 발언"을 확인하려 그 시각으로 이동했는데 다른 발언을 듣게 되고, <i>리포트가 잘못됐다고</i> 판단할 근거를 오히려 시스템이 제공하게 됩니다.                                                                                                                                                                                                                                                                                                                                                                                                                                                                           |
| 권고            | 필요한 데이터는 <b>이미 저장되어 있습니다</b> . 클라이언트가 답변 확정마다 <code>last_recording_chunk_sequence</code> 를 보내고 ( <code>api/live_handlers.py:283</code> ), 서버가 <code>session_checkpoints</code> 에 ( <code>last_final_turn_id</code> , <code>last_media_chunk_sequence</code> ) 쌍으로 기록합니다( <code>interview_service.py:265-272 · 398-405 · 463-470</code> ). 그런데 <code>runtime/worker.py</code> 와 <code>integration/interview_reporting.py</code> 의 <code>checkpoint</code> 참조가 <b>0건</b> 입니다 — ① 단기적으로는 이 체크포인트 쌍으로 턴 경계를 복원하고, ② 근본적으로는 <code>interview_turns</code> 에 시각 컬럼을 두어 시간 기준으로 조인합니다. 현재 이 테이블에는 시각 컬럼이 <b>하나도 없어</b> 시간 조인이 불가능하고, 그것이 위치 짝짓기가 나온 원인으로 보입니다. |

### F-01 리포트 생성 경로에 모델 호출이 없다

|       |                                                                                                                                                                                                                                            |
|-------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 등급    | 운영 전 필수                                                                                                                                                                                                                                    |
| 위치    | <code>workers/reporting/report.py:38</code> <code>ReportGenerator.generate</code> 및 <code>ReportRequestedEventHandler</code>                                                                                                               |
| 재현 조건 | 면접을 정상 완료하면 리포트가 생성되지만, <code>report_items.observation</code> 이 항상 "지원자의 최종 답변에서 관찰된 내용" 이고 <code>evidence.rationale</code> 은 "최종 답변의 자막과 유효 영상 구간에 직접 연결됨" 입니다. <code>model_version · prompt_version · config_version</code> 도 상수 문자열입니다. |
| 영향    | 리포트가 <b>평가 내용을 담지 못합니다</b> . 다만 Evidence 검증·연결·상태 판정(READY/PARTIAL)은 실제로 동작하므로, 골격은 완성이고 내용만 비어 있는 상태입니다.                                                                                                                                  |
| 권고    | Evidence 검증 로직은 그대로 두고 <code>observation / rationale</code> 생성만 모델 호출로 교체하면 됩니다. <code>model_version</code> 을 상수에서 실제 응답 메타데이터로 바꾸는 것을 함께 진행해야 합니다 — 그러지 않으면 어느 모델이 쓴 평가인지 추적할 수 없습니다.                                                     |

**F-02** `create_hls_job` 이 프로덕션 경로에서 호출되지 않는다

|       |                                                                                                                                                                                                                                                                                   |
|-------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 등급    | 운영 전 필수                                                                                                                                                                                                                                                                           |
| 위치    | 미디어 워커 <code>workers/reporting/media.py</code> · 어댑터의 <code>create_hls_job</code>                                                                                                                                                                                                 |
| 재현 조건 | 면접 완료 후 <code>recording_assets</code> 에 <code>.../recording/final/v1/manifest.m3u8</code> 키가 기록되지만 그 경로에 실제 매니페스트가 만들어지지 않습니다. 호출 지점은 단위 테스트에만 존재합니다.                                                                                                                             |
| 영향    | 콘솔에서 Evidence의 영상 구간을 재생하려 하면 <b>재생 자산이 없어 실패합니다</b> . 리포트의 텍스트 근거(탄·자막)는 영향을 받지 않습니다.                                                                                                                                                                                            |
| 권고    | 미디어 워커가 매니페스트 키를 기록하는 지점에서 <code>create_hls_job</code> 을 호출하고, 작업 완료를 기다린 뒤 <code>recording_assets.status</code> 를 확정하도록 연결합니다. 변환 실패 시 <code>PARTIAL</code> 로 남기는 처리도 함께 필요합니다. <b>단, F-09보다 먼저 닫지 마십시오</b> — 시각 값이 틀린 상태에서 재생만 연결하면 <b>재생은 되는데 다른 발언이 나오는</b> 상태가 되어 지금보다 나쁩니다. |

**F-03** `REPORT_GENERATING` · `REVIEWABLE` 전이를 호출하는 코드가 없다

|       |                                                                                                                                                                                              |
|-------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 등급    | 운영 전 필수                                                                                                                                                                                      |
| 위치    | <code>interview_engine/domain/session.py:68-91</code> 전이표 · 리포트 워커                                                                                                                           |
| 재현 조건 | 면접이 끝나면 세션은 <code>COMPLETED</code> 가 되고 <b>그 상태로 영구히 머무릅니다</b> . 리포트 워커는 <code>reports</code> 행을 만들지만 세션 상태를 바꾸지 않습니다. 전이표에는 두 전이가 정의돼 있습니다.                                                 |
| 영향    | "리포트가 준비됐는가"를 세션 상태로 판별할 수 없습니다. 기업 콘솔이 <code>reports</code> 테이블을 직접 조회해야 하고, 생성 중/완료를 구별할 수 없습니다.                                                                                           |
| 권고    | 리포트 워커 진입 시 <code>REPORT_GENERATING</code> , 저장 성공 시 <code>REVIEWABLE</code> 로 전이시킵니다. 전이표는 이미 이를 허용하므로 <b>호출만 추가</b> 하면 됩니다. 실패 시 재시도 중에도 <code>REPORT_GENERATING</code> 에 머무르는 것이 자연스럽습니다. |

**F-04** `hot_view_sync_status` 가 항상 `PENDING` 이다

|       |                                                                                                                                                                                                                |
|-------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 등급    | 운영 전 필수                                                                                                                                                                                                        |
| 위치    | <code>session_checkpoints.hot_view_sync_status</code> · <code>application/checkpoints.py</code>                                                                                                                |
| 재현 조건 | 체크포인트를 저장할 때 이 컬럼에 항상 <code>PENDING</code> 을 씁니다. <code>HotViewSyncStatus</code> 에는 <code>SYNCED</code> · <code>DEGRADED</code> 가 정의돼 있지만 <code>SYNCED</code> 로 바꾸는 경로는 테스트 코드에만 있습니다.                         |
| 영향    | 핫뷰 동기 상태를 <b>운영 지표로 쓸 수 없습니다</b> . "몇 %의 세션이 핫뷰 없이 진행됐나"를 이 컬럼으로 알 수 없습니다. 복구 자체는 3단 폴백으로 정상 동작하므로 <b>기능 결함은 아닙니다</b> .                                                                                        |
| 권고    | 핫뷰 <code>put</code> 성공 후 <code>SYNCED</code> , <code>HotViewUnavailable</code> 포착 시 <code>DEGRADED</code> 로 갱신합니다. <code>degraded_modes</code> 에 이미 같은 정보가 들어가므로 두 값이 어긋나지 않도록 <b>한 지점에서 함께 기록</b> 하는 것이 좋습니다. |



**F-05** `interview.checkpoint_changed` 가 큐 라우팅 표에 없다

|           |                                                                                                                                                                                                                                                                                                                                                                                                                                     |
|-----------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 등급        | 개선                                                                                                                                                                                                                                                                                                                                                                                                                                  |
| 위치        | <code>application/checkpoints.py:57</code> (발행) · <code>runtime/worker.py:51-67</code> (라우팅)                                                                                                                                                                                                                                                                                                                                        |
| 재현 조건     | 체크포인트가 갱신될 때마다 이 이벤트가 아웃박스에 적재되지만 <code>EVENT_QUEUE_ROUTING</code> 에 항목이 없어 어느 워커도 가져가지 않습니다.                                                                                                                                                                                                                                                                                                                                       |
| 영향        | <code>outbox_events</code> 에 소비되지 않는 행이 세션 진행에 비례해 쌓입니다. 기능 오류는 없지만 아웃박스 테이블이 커지고, "발행됐으니 누군가 처리한다"는 가정이 깨진 상태로 남습니다.                                                                                                                                                                                                                                                                                                               |
| 이 항목의 특이점 | 단순한 미등록이 아닙니다. <code>application/context_reconciliation.py:84-107</code> 에 이 이벤트 타입만을 처리하는 <code>reconcile_event()</code> 메서드가 이미 구현되어 있습니다 — <code>aggregate_type</code> 검증, <code>session_id</code> 와 <code>aggregate_id</code> 대조, 체크포인트 최신성 확인, <code>MissingDurableCheckpoint</code> 발생까지 갖춘 완성된 소비 로직입니다. 그런데 <code>grep -rn "reconcile_event" backend/</code> 의 결과가 정의된 파일 자신뿐입니다. 즉 발행부와 소비부가 양쪽 다 존재하는데 그 사이를 잇는 배선만 없습니다. |
| 권고        | 따라서 "발행을 지운다"는 단순 정리는 맞지 않습니다 — 지우면 이미 작성된 소비 로직도 함께 죽습니다. 둘 중 하나를 택해야 합니다 — ① <code>interview.checkpoint_changed</code> 를 라우팅 표에 추가하고 <code>reconcile_event()</code> 를 호출하는 핸들러를 <code>runtime/worker.py</code> 에 등록해 배선을 완성, ② 이 정합화 경로가 설계에서 빠졌다면 발행부와 <code>reconcile_event()</code> 를 함께 제거. 감리 권고는 ①입니다. 소비 로직이 이미 검증 조건까지 갖춰 작성돼 있다는 것은 의도된 기능이 배선 단계에서 멈춘 신호이고, 남겨두면 다음 사람이 "동작한다"고 오인할 여지가 큼니다.                            |

**F-06** `query_vector=None` 으로 매 턴 임베딩을 재계산한다

|       |                                                                                                                                                                                           |
|-------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 등급    | 개선                                                                                                                                                                                        |
| 위치    | <code>api/live_handlers.py:434</code> · <code>application/retrieval.py</code>                                                                                                             |
| 재현 조건 | 실시간 면접 중 다음 질문을 만들 때마다 <code>retrieve(query=답변텍스트, query_vector=None)</code> 으로 호출합니다. 검색 계층이 벡터를 받지 못하면 쿼리 텍스트를 다시 임베딩합니다.                                                               |
| 영향    | 턴마다 임베딩 왕복 1회가 추가됩니다. AWS Titan을 쓰는 운영 환경에서는 실시간 경로에 네트워크 왕복이 하나 더 생기는 셈이고, 지원자가 기다리는 시간이 그만큼 늘어납니다.                                                                                      |
| 권고    | 호출부에서 임베딩을 계산해 <code>query_vector</code> 로 넘깁니다. 검색 계층에는 이미 <code>len(query_vector) != 1024</code> 검사가 있어 인터페이스는 벡터를 받을 준비가 되어 있습니다. 같은 답변 텍스트로 여러 번 검색하는 경로가 있다면 턴 단위 캐시도 함께 고려할 수 있습니다. |

**F-07** FinalTurnRange.confidence 가 0.9 고정값이다

|       |                                                                                                                                                                   |
|-------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 등급    | 개선                                                                                                                                                                |
| 위치    | workers/reporting/media.py MediaRequestedEventHandler                                                                                                             |
| 재현 조건 | 미디어 워커가 자막 구간을 만들 때 FinalTurnRange(..., confidence=0.9) 로 하드코딩합니다. 실시간 인식 단계에서는 실제 confidence 를 받아 review_required: confidence < 0.75 판정에 쓰는데, 후처리에서는 그 값이 버려집니다. |
| 영향    | 리포트를 보는 사람이 음성 인식이 불확실했던 구간을 구별할 수 없습니다. 인식이 부정확한 자막을 근거로 CONFIRMED 가 나올 수 있고, 검토자는 그것을 의심할 단서를 얻지 못합니다.                                                          |
| 권고    | 실시간 단계의 confidence 를 transcript_segments.confidence 에 이미 저장하고 있으므로, 미디어 워커가 그 값을 읽어 쓰면 됩니다. 낮은 신뢰도 구간을 근거로 한 항목은 uncertainty 에 표시하는 것을 함께 권고합니다.                  |

**F-08** 기준 × 답변 턴을 zip(strict=False) 로 짝짓는다

|       |                                                                                                                                                 |
|-------|-------------------------------------------------------------------------------------------------------------------------------------------------|
| 등급    | 가장 먼저 달을 것                                                                                                                                      |
| 위치    | ReportRequestedEventHandler — zip(criterion.criteria, turns, strict=False)                                                                      |
| 재현 조건 | 평가 기준 수와 지원자 최종 답변 턴 수가 다르면 짧은 쪽 길이만큼만 항목이 만들어집니다. 예외도 로그도 없습니다. 면접이 시간 부족으로 일찍 끝나거나 답변이 무효 처리되면 실제로 발생합니다.                                     |
| 영향    | 평가되지 않은 기준이 리포트에 아예 나타나지 않습니다. 검토자는 누락을 알아챌 방법이 없고, 생성된 항목에는 모두 Evidence가 있으므로 status 도 READY 로 남습니다. 정상 리포트와 구별 불가능합니다.                        |
| 권고    | 남은 기준을 INSUFFICIENT_EVIDENCE 항목으로 명시적으로 생성하는 편이 strict=True 보다 낫습니다 — "이 기준은 답변을 얻지 못했다"가 리포트에 남아야 하는 정보이고, 예외로 리포트 생성 전체를 실패시키면 얻은 답변까지 버려집니다. |

**13.1 권고 순서**

| 순 | ID   | 작업             | 왜 이 순서인가                                                  |
|---|------|----------------|-----------------------------------------------------------|
| 1 | F-09 | 턴↔청크를 시각으로 짝짓기 | 유일하게 저장된 값이 틀린 항목. F-02보다 반드시 먼저 — 순서를 바꾸면 재생되는데 틀린 장면이 됨 |
| 2 | F-08 | 기준×답변 짝짓기 수정   | 증상이 전혀 없는 항목. 지금 닫지 않으면 나중에 왜 기준이 빠졌는지 추적 불가              |
| 3 | F-03 | 세션 상태 전이 2개 연결 | 가장 작은 변경(호출 추가만)으로 관측성이 크게 개선됨. 이후 작업의 진행 확인에도 필요         |
| 4 | F-01 | 리포트 모델 연동      | 제품의 핵심 가치. 단 Evidence 골격이 완성돼 있어 교체 범위가 좁습니다              |
| 5 | F-02 | HLS 변환 연결      | F-09 이후에. F-01로 리포트 내용이 생긴 뒤에야 영상 재생 수요가 실제로 발생           |
| 6 | F-07 | 자막 신뢰도 전달      | F-01·F-09와 같은 코드 영역( FinalTurnRange ). 함께 처리하면 효율적        |
| 7 | F-04 | 핫뷰 동기 상태 기록    | 운영 관측 항목. 실제 트래픽이 생기기 전에                                  |
| 8 | F-06 | 임베딩 재사용        | 성능 항목. Titan 연동 후 실측하고 판단해도 늦지 않음                         |
| 9 | F-05 | 체크포인트 이벤트 배선   | 가장 가벼움. reconcile_event() 가 이미 있으므로 배선 완성을 권고             |

## 14 감리 판정

개발 단계 산출물로서 **적합**. 단 **F-09는 미완성이 아닌 결함**이며, 실제 채용 사용 전에 달아야 할 항목이 6건입니다.

### 14.1 구간별 판정

| 구간                      | 판정    | 근거                                                                                                                                                                              |
|-------------------------|-------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 면접 전 제출물 분석<br>LANE B   | 동작    | 업로드 2단 검증 · 소유권 3단 판정 · 검증 타킷 5종 분류 · 정렬과 시간 예산까지 구현. 소유권 점수 상한 0.9와 <code>OWNERSHIP_UNCERTAIN</code> 처리가 <b>단정하지 않는 설계</b> 를 보여줌                                               |
| 실시간 면접<br>LANE C        | 동작    | 음성·영상 경로 분리 · 5개 답변 가드 · 정책 검사 5종 · 환각 출력 필터 · 3단 복구 폴백 모두 구현. F-06(임베딩 재계산)만 성능 항목으로 남음                                                                                        |
| 면접 후 리포트<br>LANE D      | 결함 포함 | 이벤트 체인 · 재시도 안전성 · Evidence 검증 골격은 완성. <b>평가 내용 생성(F-01)과 영상 재생(F-02)이 비어 있고, 영상 시각 값이 틀립니다(F-09)</b> . F-08은 조용한 누락 위험                                                         |
| 영상 분석                   | 해당 없음 | 표정·시선·자세 분석은 <b>존재하지 않으며, 설계상 들어갈 자리도 없음</b> . Evidence 구조와 검증 타킷 5종이 모두 발언 내용 기반. 미구현이 아니라 선택으로 판단                                                                             |
| 테넌시 · 개인정보<br>LANE A 공통 | 양호    | 업무 테이블 49/50에 <code>company_id</code> (예외는 소비 기록용 <code>processed_messages</code> ) · 리포지토리 SQL 강제 · 404 응답 · 토큰 해시 저장 · 본문 객체 저장소 분리 · 삭제 검증 ( <code>verified_at</code> )까지 일관 |

### 14.2 운영 전환 전 필수 조건 6건

1. **F-09** 턀↔청크를 시각으로 째짓기 — 반드시 **F-02보다 먼저** (유일하게 값이 틀린 항목)
2. **F-08** 기준×답변 째짓기 — 누락이 리포트에 드러나게
3. **F-01** 리포트 평가 내용 모델 연동 + `model_version` 을 실제 값으로
4. **F-02** HLS 변환 연결 — Evidence 구간 재생 가능하게 (F-09 이후)
5. **F-03** 세션 상태 전이 2개 — 리포트 준비 상태를 관측 가능하게
6. **F-04** 핫뷰 동기 상태 기록 — 저하 비율을 측정 가능하게

#### 감리 총평

이 시스템의 핵심 주장은 "AI가 만든 모든 판단은 지원자의 실제 답변으로 되돌아갈 수 있다"입니다. 감리 결과, 그 주장을 뒷받침하는 구조는 실제로 구현되어 있습니다.

- `evidence` 가 답변 턀 ID · 자막 세그먼트 ID · 영상 구간을 **동시에** 요구하고, 검증 실패 시 `CONFIRMED` 가 되지 않습니다 (단 그 영상 시각 값은 F-09로 아직 틀립니다)
- 질문의 근거가 `question_source_references` 로 별도 기록되고, 모델이 만들어낸 가짜 출처는 코드가 잘라냅니다
- 기준 버전 ID가 초대→전략→세션→리포트에 **동일하게** 박혀 평가 기준의 사후 변경을 무력화합니다
- 사람의 수정이 AI 원본을 덮어쓰지 않고 `human_reviews` 에 누적됩니다
- 영상은 판단의 입력이 아니라 **검증 수단**으로만 쓰입니다

비어 있는 것은 대부분 그 구조 안에 들어갈 내용입니다. Evidence를 검증하는 코드는 동작하는데 Evidence에 붙일 관찰 문장을 모델이 쓰지 않고, 영상 구간을 재생할 자산이 만들어지지 않습니다.

순서로서는 옳습니다. 근거 검증 골격 없이 모델을 먼저 붙였다면 그럴듯한 문장이 근거 없이 저장되는 시스템이 됐을 것이고, 나중에 근거를 강제하려면 전체를 다시 만들어야 했을 것입니다. 지금은 모델을 붙이면 그 출력이 자동으로 근거 검증을 받는 상태입니다.

단 하나, 위 주장을 실제로 깨는 항목이 F-09입니다. "실제 답변으로 되돌아갈 수 있다"의 마지막 한 걸음(영상에서 몇 초인지)이 턀를 청크와 위치로 째지어 정해지고 있고, 검증조차 그 잘못된 값을 기준으로 삼아 통과시킵니다. 구조가 옳기 때문에 고칠 범위는 좁습니다 — 필요한 데이터( `session_checkpoints` 의 턀↔청크 번호 쌍)는 이미 저장되어 있고, 미디어 워커가 그것을 읽기만 하면 됩니다. F-08도 같은 성격(조용한 오류)이므로 두 항목을 함께 달는 것을 권고합니다.

### 14.3 감리 근거 자료

| 항목     | 내용                                                                                                                                                |
|--------|---------------------------------------------------------------------------------------------------------------------------------------------------|
| 기준 커밋  | 7c998be · main · <code>git status --porcelain</code> 0줄                                                                                           |
| 테스트 실효 | uv run --no-sync pytest backend/tests → 193 passed · 1 skipped · 7.13s                                                                            |
| 열람한 소스 | 4개 레인 × (adapters · api · application · domain · repositories) · shared · runtime · workers · apps 2개                                             |
| ERD 근거 | company_management/repositories/postgres.py (12) · submission_analysis (12) · interview_engine (8+1) · reporting (10) · shared/persistence.py (7) |
| 다이어그램  | 인라인 SVG 15장(ERD 5 · 시퀀스 4 · 구조 6) · 외부 스크립트 없음 · 오프라인·인쇄 동작                                                                                       |
| 감리 일자  | 2026-08-16                                                                                                                                        |